# DBMS

## Complete Guide

Database Management Systems

Placement Preparation -- Full Reference

---

**Table of Contents**

# ■■■■■ Chapter 1: DBMS Fundamentals & Architecture

Welcome to the ultimate placement-focused guide to DBMS Fundamentals. We are going to break down complex database concepts using simple analogies, mind maps, and memory tricks.

## 1. What is Data, Information, and Database?

> *[!NOTE]*
>
> *Think of a Database like a **massive, highly-organized library vault**.*
>
> *Books are the **data**, the catalog is the **schema**, and the librarian who helps you find, add, or remove books is the **DBMS**.*

- **Data:** Raw, unorganized facts. (e.g., `101`, `Alice`, `22`)
- **Information:** Processed, organized data that makes sense. (e.g., "Alice is a 22-year-old student with ID 101")
- **Database:** A database is an organized collection of related data that can be easily stored, accessed, managed, and updated.
- **DBMS (Database Management System):** Database Management System (DBMS) is software used to create, store, retrieve, update, and manage data in a database. It acts as an interface between users/applications and the database while ensuring efficient data management, security, consistency, and integrity.
  - *Examples:* MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server.

### ■ Mind Map: The Database Ecosystem

```
TEXT
       [ End Users ]       [ Applications ]
             \                   /
              \                 /
               v               v
     +---------------------------------+
     |          DBMS Software          | <-- (The Librarian)
     | (MySQL, Oracle, PostgreSQL)     |
     +---------------------------------+
                      |
                      v
     +---------------------------------+
     |           DATABASE              | <-- (The Library Vault)
     |  [Tables] [Views] [Indexes]     |
     +---------------------------------+
```

## 2. File System vs. DBMS (Why do we need a DBMS?)

Before DBMS, we saved data in flat files (like `.txt` or `.csv`). This caused massive headaches.

> *[!WARNING]*
>
> ***The Problem with File Systems:*** *Imagine trying to find the bank balance of one specific person in a text file containing 10 million lines. It would take forever!*

### ■ Comparison Table: File System vs DBMS

| Feature | File System | DBMS |
|---|---|---|
| Data Redundancy | High (Same data saved multiple times) | Low (Controlled by Normalization) |
| Data Inconsistency | High (Updating one file doesn't update others) | Low (Centralized control) |
| Data Searching | Slow and difficult | Extremely fast (using Indexes) |
| Security | Very low (Anyone can open a file) | High (Passwords, Roles, Permissions) |
| Concurrent Access | Poor (Two people editing a file crashes it) | Excellent (Transactions and Locking) |
| Crash Recovery | None (If computer crashes while saving, data is lost) | High (ACID properties and logs) |

---

## ■■■■■ 3. The 3-Level Architecture of DBMS (Most Asked Interview Question)

The 3-Level Architecture is used to provide Data Abstraction, Data Independence, Security, and easier database management.

> *[!TIP]*
>
> *### ■ Quick Interview Answer*
>
> *"The 3-Level Architecture divides a database into External, Conceptual, and Internal levels. Its purpose is to provide data abstraction, data independence, security, and easier database management."*

### The Three Levels Explained

• **External Level:** What users see. It includes the views that hide the rest of the database from the end-user for security and simplicity.

• **Conceptual Level:** Logical structure of the database. It defines what data is stored and the relationships among those data (Tables, Constraints).

• **Internal Level:** Physical storage details. It defines how the data is actually stored on the physical storage devices (B-Trees, Hashing, data compression).

> *[!NOTE]*
>
> *### ■ Memory Trick*
>
> ** **External** = User View*
>
> ** **Conceptual** = Database Design*
>
> ** **Internal** = Physical Storage*

### ■ Real-World Analogy: The Restaurant

- **External Level (The Menu):** What the customer sees. You only see the names of the dishes and prices.
- **Conceptual Level (The Recipe Book):** What the chef sees. The ingredients and relationships between them.
- **Internal Level (The Kitchen Storage):** Where the ingredients are physically stored on the shelves.

## ■■ Architecture Diagram

```
TEXT
[User 1]   [User 2]   [User 3]     <-- End Users
   |          |          |
   v          v          v
+-------+  +-------+  +-------+
| View 1|  | View 2|  | View 3|    <-- EXTERNAL LEVEL (What users see)
+-------+  +-------+  +-------+
       \      |      /
        \     |     /
         v    v    v
+----------------------------+
|      CONCEPTUAL LEVEL      |    <-- (Tables, Relations, Constraints)
+----------------------------+
              |
              v
+----------------------------+
|       INTERNAL LEVEL       |    <-- (Physical Storage Details)
+----------------------------+
              |
              v
     [( Physical Database )]       <-- Hard Drive / SSD
```

## ■ Important Interview Note: Data Independence

### ■ Easy Definition

Data Independence means making changes in one part of the database without affecting other parts.

### ■ Interview Definition

Data Independence is the ability to change the schema at one level of the database without affecting the schema at the next higher level.

### 1■■ Physical Data Independence

### ■ Easy Definition

Changes in how data is stored should not affect the database design.

### ■ Example

Moving data from HDD to SSD or changing an index should not affect tables or applications.

### ■ Memory Trick

*Storage Changes → Design Unaffected*

### 2■■ Logical Data Independence

### ■ Easy Definition

Changes in database design should not affect users or applications.

■ **Example**

Adding a new column to a table should not break existing user views.

■ **Memory Trick**

*Design Changes → User Unaffected*

> *[!TIP]*
>
> ■ ***Interview Fact***
>
> *Physical Data Independence is easier to achieve than Logical Data Independence.*

---

## 4. Keys in DBMS (The Most Important Foundation)

Keys are used to uniquely identify a row in a table or to establish a relationship between tables.

> *[!TIP]*
>
> ***Analogy:***
>
> *A **Candidate Key** is anyone who can be the captain of a team.*
>
> *The **Primary Key** is the person actually chosen to be the captain.*
>
> *An **Alternate Key** is anyone who could have been captain but wasn't chosen.*

### Super Key

A single key or a group of keys that uniquely identify a row.

- *Example:* `{Emp_ID}`, `{Emp_ID, Email}`, `{Emp_ID, Name}`.
- **Important:** Every table has at least one Super Key (the combination of all columns).

### Candidate Key

A minimal Super Key. It uniquely identifies a row, but it has no unnecessary attributes.

- *Example:* `{Emp_ID}` is a candidate key. `{Email}` is a candidate key. `{Emp_ID, Name}` is NOT a candidate key because `Name` is unnecessary.

### Primary Key (PK)

The one Candidate Key chosen by the Database Administrator to uniquely identify rows.

- **Rules:** Cannot be NULL. Cannot be duplicate. Only ONE Primary Key per table.

### Alternate Key

The Candidate Keys that were NOT chosen to be the Primary Key.

- *Example:* If `{Emp_ID}` is chosen as the PK, then `{Email}` becomes an Alternate Key.

### Foreign Key (FK)

A key used to link two tables together. It is an attribute in one table that refers to the Primary Key in another table.

### ■■ Relationship Diagram: Primary Key vs Foreign Key

```
TEXT
Table 1: DEPARTMENT                Table 2: EMPLOYEE
+---------+-------------+          +--------+-----------+---------+
| Dept_ID | Dept_Name   |          | Emp_ID | Emp_Name  | Dept_ID |
+---------+-------------+          +--------+-----------+---------+
|   10    | HR          |<----     |  1     | Alice     |   10    |
|   20    | IT          |    |     |  2     | Bob       |   20    |
+---------+-------------+    |     |  3     | Charlie   |   10    |
      ^                      |     +--------+-----------+---------+
      |                      |                              ^
[Primary Key]                +------------------------------|
                                                  [Foreign Key]
```

---

## 5. More Key Types (Composite, Surrogate, Natural)

### Composite Key

A primary key made up of **two or more columns** that together uniquely identify a row. Neither column alone is unique.

**Example:**

`Enrollment(Student_ID, Course_ID)` — Neither `Student_ID` nor `Course_ID` alone is unique, but the combination is.

```
SQL
CREATE TABLE Enrollment (
    Student_ID INT,
    Course_ID  INT,
    Grade      CHAR(1),
    PRIMARY KEY (Student_ID, Course_ID)   -- Composite Primary Key
);
```

### Surrogate Key

An **artificial key** created by the system (usually an auto-incremented integer) when no good natural key exists.

- **Natural Key:** Uses real-world data (e.g., `Aadhaar_Number`, `Email`).
- **Surrogate Key:** Meaningless number assigned by the DB (e.g., `Emp_ID = 101`).

> *[!TIP]*
>
> ***Interview Answer:*** *Surrogate keys are preferred because natural keys can change (e.g., a person changes their email) which would break all foreign key references. Surrogate keys never change.*

---

## 6. Integrity Constraints

Integrity constraints are rules enforced by the DBMS to ensure accuracy and consistency of data.

| Constraint | Rule | Example |
|---|---|---|
| Entity Integrity | Primary Key cannot be NULL | Emp_ID must always have a value |
| Referential Integrity | Foreign Key must match an existing PK or be NULL | Dept_ID in Employee must exist in Department |
| Domain Integrity | Values must belong to a valid domain (data type, range) | Age must be a positive integer |
| User-Defined Integrity | Custom business rules | Salary must be > 0, Grade must be A/B/C/D/F |

```sql
SQL
CREATE TABLE Employees (
    Emp_ID   INT         PRIMARY KEY,              -- Entity Integrity
    Emp_Name VARCHAR(50) NOT NULL,                 -- Domain Integrity
    Salary   DECIMAL(10,2) CHECK (Salary > 0),     -- User-Defined Integrity
    Dept_ID  INT,
    FOREIGN KEY (Dept_ID) REFERENCES Departments(Dept_ID)  -- Referential Integrity
);
```

### Referential Integrity Actions (What happens when a PK is deleted?)

| Action | Behavior |
|---|---|
| ON DELETE CASCADE | Deletes all child rows automatically |
| ON DELETE SET NULL | Sets FK to NULL in child rows |
| ON DELETE RESTRICT | Blocks deletion if child rows exist (default) |
| ON DELETE NO ACTION | Similar to RESTRICT but checked at end of transaction |

```sql
SQL
-- Example: Deleting a Department auto-deletes its employees
FOREIGN KEY (Dept_ID) REFERENCES Departments(Dept_ID) ON DELETE CASCADE
```

---

## 7. Functional Dependencies (FD)

A **Functional Dependency** (written as X → Y) means that knowing X uniquely determines the value of Y.

> *[!NOTE]*
>
> ***Analogy:*** `Student_ID` → `Student_Name` *means "if you know the Student ID, you will always know exactly one Student Name." The ID determines the Name.*

### Types of Functional Dependencies

---

| Type | Definition | Example |
|---|---|---|
| Trivial FD | Y is a subset of X. (X → Y where Y ⊆ X) | {A, B} → A |
| Non-Trivial FD | Y is NOT a subset of X | Student_ID → Name |
| Partial FD | Y depends on part of a composite PK | {SID, CID} → SName (only SID determines SName) |
| Transitive FD | X → Y → Z (Z depends on a non-key Y) | EmpID → DeptID → DeptName |
| Multi-valued FD | X →→ Y (X determines a set of Y values) | Student →→ Phone (student has many phones) |

### Armstrong's Axioms (Rules to Derive New FDs)

These are the foundational rules for reasoning about functional dependencies.

| Axiom | Rule | Example |
|---|---|---|
| Reflexivity | If Y ⊆ X, then X → Y | {A, B} → A |
| Augmentation | If X → Y, then XZ → YZ | If A → B, then AC → BC |
| Transitivity | If X → Y and Y → Z, then X → Z | If A → B and B → C, then A → C |

> *[!TIP]*
>
> ***Derived Rules:***
>
> *- **Union:** If X→Y and X→Z, then X→YZ*
>
> *- **Decomposition:** If X→YZ, then X→Y and X→Z*
>
> *- **Pseudotransitivity:** If X→Y and WY→Z, then WX→Z*

### Finding Closure (X■) — How to find all attributes determined by X

**Algorithm:** Given FD set F and set of attributes X, find X■:

1. Start with X■ = X
2. For each FD A→B in F: if A ⊆ X■, add B to X■
3. Repeat until no new attributes are added

**Example:**

```
Given: FDs = { A→B, B→C, C→D }
Find: A■

Step 1: A■ = {A}
Step 2: A→B applies (A⊆{A}), so A■ = {A, B}
Step 3: B→C applies (B⊆{A,B}), so A■ = {A, B, C}
Step 4: C→D applies, so A■ = {A, B, C, D}
Result: A■ = {A, B, C, D} → A is a Super Key!
```

---

## 8. Database Languages (SQL Categories)

SQL is divided into sub-languages based on their function.

> *[!TIP]*
>
> *### ■ Memory Trick: The Acronyms*
>
> ** **DDL (Define):** Building the house (CREATE, ALTER, DROP, TRUNCATE).*
>
> ** **DML (Manipulate):** Moving furniture inside the house (INSERT, UPDATE, DELETE).*
>
> ** **DQL (Query):** Looking inside the house (SELECT).*
>
> ** **TCL (Transaction):** Saving your progress in a video game (COMMIT, ROLLBACK, SAVEPOINT).*
>
> ** **DCL (Control):** Giving the house key to someone (GRANT, REVOKE).*

### ■ Process Flow Diagram: DDL vs DML vs DQL

```
TEXT
  [DDL] -> Creates the Table Structure (The Empty Box)
    |
    v
  [DML] -> Inserts/Updates Data (Puts items in the Box)
    |
    v
  [DQL] -> Selects Data (Reads items from the Box)
```

### Important Difference: DELETE vs DROP vs TRUNCATE (■■■■■)

This is asked in almost every technical interview!

| Feature | DELETE | TRUNCATE | DROP |
|---|---|---|---|
| Language Type | DML | DDL | DDL |
| What it does? | Deletes rows one by one. | Deletes all rows at once. | Deletes table structure + data. |
| Rollback? | Can be rolled back. | Cannot be rolled back. | Cannot be rolled back. |
| WHERE clause? | Yes, you can use WHERE. | No, empties the entire table. | No, destroys the entire table. |
| Speed | Slow (logs each row). | Very Fast (deallocates pages). | Very Fast. |

---

# ■ CHAPTER END REVISIONS ■

## ■ 5-Minute Quick Revision

1. **Data vs Info:** Data is raw facts; Information is processed data.
2. **File System Issues:** Data redundancy, inconsistency, slow access, poor security.
3. **3-Level Architecture:** External (views), Conceptual (tables/logic), Internal (physical storage).
4. **Data Independence:** Ability to modify one level without affecting the level above it.

5. **Keys:** Super (unique combinations), Candidate (minimal super), Primary (chosen candidate), Foreign (links tables).
6. **Composite Key:** PK made of 2+ columns. Surrogate Key: system-generated artificial key.
7. **Languages:** DDL (CREATE/DROP), DML (INSERT/UPDATE), DQL (SELECT), TCL (COMMIT/ROLLBACK), DCL (GRANT/REVOKE).
8. **TRUNCATE vs DELETE:** TRUNCATE is fast, DDL, no rollback. DELETE is slower, DML, can rollback.
9. **Integrity Constraints:** Entity (PK not NULL), Referential (FK matches PK), Domain (valid data type), User-defined (CHECK).
10. **Functional Dependency X→Y:** Knowing X uniquely determines Y.
11. **Armstrong's Axioms:** Reflexivity, Augmentation, Transitivity.
12. **Closure X■:** All attributes that can be derived from X using FD rules.
13. **Referential Actions:** CASCADE (auto-delete children), SET NULL, RESTRICT (block delete).

## ■ Common Mistakes Students Make in Interviews

1. **Confusing Super Key and Candidate Key:** A Candidate Key must be *minimal*. A Super Key doesn't have to be.
2. **Saying TRUNCATE is DML:** TRUNCATE is **DDL**. It does not log individual row deletions.
3. **Saying Foreign Key cannot be NULL:** A Foreign Key **CAN** be NULL (e.g., an employee who hasn't been assigned a department yet). It is Primary Keys that cannot be NULL.
4. **Confusing Partial and Transitive FD:** Partial FD requires a composite primary key and a non-key depending on part of it. Transitive FD is a chain: X → non-key → another non-key.
5. **Forgetting ON DELETE actions:** Many students describe referential integrity but forget to mention CASCADE, SET NULL, RESTRICT options when asked about FK behavior.
6. **Saying Surrogate Key has real meaning:** Surrogate keys are intentionally meaningless. Their value conveys nothing about the entity.

## ■ Top 10 Placement MCQs

**Q1. Which level of architecture provides Physical Data Independence?**

A) External Level

B) Conceptual Level

C) Internal Level

D) Logical Level

> ***Answer: B) Conceptual Level.***
>
> *Physical Data Independence is the ability to change the **Internal schema** (physical storage) without requiring changes to the **Conceptual schema**. The Conceptual Level acts as the shield — it remains unaffected when storage details change. This independence is achieved through the **Conceptual/Internal mapping layer**. Note: Logical Data Independence (Q5 above) is the ability to change the Conceptual schema without affecting the External/user level.*

**Q2. Which of the following is NOT a DML command?**

A) INSERT

B) UPDATE

C) TRUNCATE

D) DELETE

> ***Answer: C) TRUNCATE.*** *(TRUNCATE is a DDL command).*

**Q3. A minimal Super Key is called a:**

A) Primary Key

B) Foreign Key

C) Candidate Key

D) Alternate Key

> ***Answer: C) Candidate Key.***

**Q4. Can a Foreign Key accept NULL values?**

A) Yes, always.

B) No, never.

C) Yes, unless a NOT NULL constraint is explicitly applied.

D) Only if the Primary Key it references is also NULL.

> ***Answer: C.*** *(Foreign keys can be NULL).*

**Q5. The ability to modify the conceptual schema without altering the external schema is called:**

A) Physical Data Independence

B) Logical Data Independence

C) Architecture Independence

D) View Independence

> ***Answer: B) Logical Data Independence.***

**Q6. Which integrity constraint ensures the Primary Key column can never be NULL?**

A) Domain Integrity

B) Referential Integrity

C) Entity Integrity

D) User-defined Integrity

> ***Answer: C) Entity Integrity.***

**Q7. Given FDs: A→B and B→C. What is A■ (closure of A)?**

A) {A}

B) {A, B}

C) {A, B, C}

D) {B, C}

> ***Answer: C) {A, B, C}.*** *By transitivity: A→B→C, so A determines B and C.*

**Q8. Which Armstrong's Axiom states: If X→Y, then XZ→YZ?**

A) Reflexivity

B) Augmentation

C) Transitivity

D) Decomposition

> ***Answer: B) Augmentation.***

**Q9. A key made up of two or more columns that together uniquely identify a row is called a:**

A) Candidate Key

B) Surrogate Key

C) Composite Key

D) Alternate Key

> ***Answer: C) Composite Key.***

**Q10. When a parent record is deleted and the `ON DELETE CASCADE` rule is applied, what happens to child records?**

A) Child records are set to NULL

B) Child records are blocked from deletion

C) Child records are automatically deleted

D) An error is thrown

> ***Answer: C) Child records are automatically deleted.***

## ■ Top 10 Interview Questions

1. **Explain the difference between DELETE, TRUNCATE, and DROP.**
   • *Answer:* Mention DML vs DDL, ability to Rollback, speed, and whether the table structure remains.
2. **What is the difference between Logical and Physical Data Independence?**
   • *Answer:* Logical allows changing the conceptual schema (adding columns) without affecting external views. Physical allows changing storage structures (like adding an SSD or changing an index) without affecting the conceptual schema.
3. **What is a Foreign Key? Can a table have multiple Foreign Keys?**
   • *Answer:* Yes, a table can have multiple foreign keys pointing to different tables. It establishes a relationship between two tables.
4. **Why do we need a Candidate Key if we already have a Primary Key?**
   • *Answer:* A table might have multiple columns that uniquely identify a row (e.g., Email, SSN, Employee_ID). All of these are Candidate Keys. The DBA chooses one to be the Primary Key. The rest are Alternate Keys.
5. **What happens to the Foreign Key data if the Primary Key record is deleted?**
   • *Answer:* It depends on the constraint. It could block the deletion (`RESTRICT`), delete the foreign key records too (`CASCADE`), or set the foreign key to NULL (`SET NULL`).
6. **What is a Functional Dependency? Give an example.**
   • *Answer:* A FD `X→Y` means knowing the value of X uniquely determines the value of Y. Example: `Employee_ID → Employee_Name` means knowing the Employee ID always gives you exactly one Employee Name.
7. **What are Armstrong's Axioms? Why are they important?**
   • *Answer:* They are inference rules (Reflexivity, Augmentation, Transitivity) used to derive all valid functional dependencies from a given set. They form the mathematical foundation of normalization

theory.

8. **What is the difference between a Natural Key and a Surrogate Key?**
   • *Answer:* A Natural Key is derived from real-world data (like Email or Aadhaar). A Surrogate Key is an artificial, system-generated identifier (like an auto-increment ID) with no business meaning. Surrogate keys are preferred because natural keys can change, breaking FK references.
9. **Explain Entity Integrity and Referential Integrity.**
   • *Answer:* Entity Integrity: The Primary Key of a table cannot be NULL or duplicate. Referential Integrity: Every Foreign Key value must either be NULL or match an existing Primary Key value in the referenced table.
10. **How do you find the closure of a set of attributes?**
    • *Answer:* Start with X■ = X. Repeatedly apply FDs: if the left-hand side of any FD is a subset of X■, add the right-hand side to X■. Continue until no more attributes can be added. The result X■ is the closure.

# ■■■■■ Chapter 2: ER Model & Normalization

This chapter covers the database design process. We will visually map out requirements using the ER Model and then eliminate data anomalies using Normalization. This is the **most heavily tested** topic in technical interviews.

---

## 1. Entity-Relationship (ER) Model

The ER model is a blueprint of your database. Before writing any SQL, you draw an ER diagram.

### ■ Mind Map: ER Model Components

```
TEXT
          [ ER MODEL ]
         /      |     \
        /       |      \
   [Entity] [Attribute] [Relationship]
    (Noun)  (Adjective)     (Verb)
```

> *[!NOTE]*
>
> ***Analogy:***
>
> ***Entity:*** *A Student (Person, Place, Object).*
>
> ***Attribute:*** *Student's Name, Age, ID (Characteristics of the entity).*
>
> ***Relationship:*** *Student enrolls in a Course (How entities interact).*

### ■■ ASCII ER Diagram Example

```
TEXT
  +---------+          +----------+          +--------+
  | Student | ===(1)===<  Enrolls  >===(M)=== | Course |
  +---------+          +----------+          +--------+
       |                                         |
   +---+---+                                 +---+---+
   |       |                                 |       |
(ID)     (Name)                          (C_ID)  (C_Name)
```

### Relationship Types (Cardinality)

1. **One-to-One (1:1):** One Citizen has One Passport.
2. **One-to-Many (1:M):** One Department has Many Employees. (Most common).
3. **Many-to-Many (M:N):** Many Students enroll in Many Courses.

---

## 2. What is Normalization?

Normalization is the process of organizing data to reduce **redundancy** (duplication) and improve data integrity.

> *[!WARNING]*
>
> ***Why do we Normalize? (The 3 Anomalies)***
>
> *1. **Insertion Anomaly:** Cannot insert data because other data is missing. (e.g., Cannot add a new Course if no Student is enrolled in it yet).*
>
> *2. **Update Anomaly:** Updating a value requires changing it in 100 different rows. If you miss one, data becomes inconsistent.*
>
> *3. **Deletion Anomaly:** Deleting a row accidentally deletes unrelated data. (e.g., Deleting the last student in a course deletes the course details entirely).*

---

## 3. Step-by-Step Normalization (1NF to BCNF)

Let's walk through the exact steps to normalize a database, exactly as expected in product-based company interviews.

### ■ First Normal Form (1NF)

**Rule:** Attributes must be atomic (single-valued). No multi-valued attributes or nested tables.

**Original Table:**

| Emp_ID | Emp_Name | Phone_Numbers |
|---|---|---|
| 1 | Alice | 555-1111, 555-2222 |
| 2 | Bob | 555-3333 |

**The Problem:** Alice has two phone numbers in one cell. This makes searching for a specific phone number very difficult.

**The Fix (Decomposition):** Create a new row for every multi-valued attribute.

**Final 1NF Table:**

| Emp_ID | Emp_Name | Phone_Number |
|---|---|---|
| 1 | Alice | 555-1111 |
| 1 | Alice | 555-2222 |
| 2 | Bob | 555-3333 |

---

### ■■ Second Normal Form (2NF)

**Rule:** Must be in 1NF **AND** no Partial Dependencies.

*(Partial Dependency: A non-prime attribute depends on only a PART of a composite primary key).*

**Original 1NF Table:** `Student_Course` (Primary Key = `{Student_ID, Course_ID}`)

| Student_ID | Course_ID | Student_Name | Course_Fee |
|---|---|---|---|
| 101 | C1 | John | $500 |
| 101 | C2 | John | $300 |

**The Problem:** `Student_Name` only depends on `Student_ID` (part of the PK). `Course_Fee` only depends on `Course_ID` (part of the PK). This causes redundancy — John's name is stored once per course he enrolls in.

**The Functional Dependencies (Violations):**

- `{Student_ID}` → `Student_Name` ← **Partial Dependency** (Student_Name needs only Student_ID, not the full composite PK)
- `{Course_ID}` → `Course_Fee` ← **Partial Dependency** (Course_Fee needs only Course_ID, not the full composite PK)

> *[!IMPORTANT]*
>
> *Since non-prime attributes (`Student_Name`, `Course_Fee`) depend on only a **part** of the composite Primary Key `{Student_ID, Course_ID}`, this table **violates 2NF**.*

**The Fix (Decomposition):** Break it into 3 tables based on what defines what.

**Final 2NF Tables:**

- **Student Table:** (PK: `Student_ID`)

| | Student_ID | Student_Name |
|---|---|---|
| | 101 | John |

- **Course Table:** (PK: `Course_ID`)

| | Course_ID | Course_Fee |
|---|---|---|
| | C1 | $500 |
| | C2 | $300 |

- **Enrollment Table:** (PK: `{Student_ID, Course_ID}`)

| | Student_ID | Course_ID |
|---|---|---|
| | 101 | C1 |
| | 101 | C2 |

**Why it worked:** No more partial dependencies. John's name is stored only once.

---

## ■■■ Third Normal Form (3NF)

**Rule:** Must be in 2NF **AND** no Transitive Dependencies.

*(Transitive Dependency: A non-prime attribute depends on another non-prime attribute).*

**Original 2NF Table:** `Employee` (Primary Key = `Emp_ID`)

| Emp_ID | Emp_Name | Dept_ID | Dept_Name |
|---|---|---|---|
| 1 | Alice | D1 | HR |
| 2 | Bob | D1 | HR |

**The Problem:** `Dept_Name` depends on `Dept_ID`. `Dept_ID` depends on `Emp_ID`. Therefore, `Dept_Name` transitively depends on `Emp_ID`. HR is repeated for every HR employee.

**The Dependency:** `Emp_ID -> Dept_ID -> Dept_Name`

**The Fix (Decomposition):** Move the transitively dependent attributes to a new table.

**Final 3NF Tables:**

- **Employee Table:**

| | Emp_ID | Emp_Name | Dept_ID (FK) |
|---|---|---|---|
| | 1 | Alice | D1 |
| | 2 | Bob | D1 |

- **Department Table:**

| | Dept_ID | Dept_Name |
|---|---|---|
| | D1 | HR |

**Why it worked:** If the HR department changes its name to Human Resources, we only update it in ONE place (the Department table).

---

## ■■■■ Boyce-Codd Normal Form (BCNF)

**Rule:** Must be in 3NF **AND** for every functional dependency `X -> Y`, `X` must be a Super Key.

*(Simply: If A determines B, A better be a key!)*

> *[!IMPORTANT]*
>
> ***Difference between 3NF and BCNF:*** *3NF allows `X -> Y` if `Y` is a prime attribute (part of a candidate key). BCNF is stricter and does not allow this exception.*

**Original 3NF Table:** `Student_Subject_Professor`

*(Rule: A student can take multiple subjects. Each subject has multiple professors. A professor teaches ONLY ONE subject.)*

Primary Key: `{Student, Subject}`

| Student | Subject | Professor |
|---|---|---|
| S1 | Math | Prof_A |

---

| Student | Subject | Professor |
|---|---|---|
| S2 | Math | Prof_B |
| S3 | Math | Prof_A |

**The Problem:** The table is in 3NF. However, `Professor -> Subject` (because a professor teaches only one subject). But `Professor` is NOT a super key (Prof_A appears twice). This is a BCNF violation.

**The Fix (Decomposition):**

**Final BCNF Tables:**

- **Professor_Subject Table:** (PK: `Professor`)

| | Professor | Subject |
|---|---|---|
| | Prof_A | Math |
| | Prof_B | Math |

- **Student_Professor Table:** (PK: `{Student, Professor}`)

| | Student | Professor |
|---|---|---|
| | S1 | Prof_A |
| | S2 | Prof_B |
| | S3 | Prof_A |

**Why it worked:** Now, every determinant (left side of the arrow) is a Super Key.

---

## ■■■■■ Fourth Normal Form (4NF)

**Rule:** Must be in BCNF **AND** no non-trivial Multi-Valued Dependencies (MVD) exist.

A **Multi-Valued Dependency (MVD)** $X \twoheadrightarrow Y$ means: for a given X, there exists a *set* of Y values that are independent of any other attribute Z.

**Original Table:** `Student_Skills_Languages`

*(A student can have multiple skills AND multiple languages. These are independent facts.)*

| Student | Skill | Language |
|---|---|---|
| Alice | Python | English |
| Alice | Python | Hindi |
| Alice | SQL | English |
| Alice | SQL | Hindi |

**The Problem:** Every combination of Alice's skills and languages must be stored. Adding a new language requires adding rows for EVERY skill, causing massive redundancy.

**The Fix (Decomposition):**

---

- **Student_Skill Table:** (Alice, Python), (Alice, SQL)
- **Student_Language Table:** (Alice, English), (Alice, Hindi)

> *[!IMPORTANT]*
>
> ***When does 4NF apply?*** *Only when a table has two or more independent multi-valued attributes for the same primary key. This is fairly rare but asked in advanced interviews.*

---

### ■■■■■ Fifth Normal Form (5NF) / Project-Join Normal Form (PJNF)

**Rule:** Must be in 4NF **AND** every join dependency is implied by the candidate keys.

5NF deals with **Join Dependencies**: a table cannot be reconstructed by joining smaller tables without losing or adding data.

> *[!TIP]*
>
> *5NF is rarely discussed in placements beyond the definition. The key answer: "A table is in 5NF if it cannot be decomposed further without losing information." This is the highest practical normal form.*

---

### Summary: Normal Forms at a Glance

```
MERMAID
graph TD
    UF[Unnormalized] --> NF1[1NF: Atomic Values]
    NF1 --> NF2[2NF: No Partial FDs]
    NF2 --> NF3[3NF: No Transitive FDs]
    NF3 --> BCNF[BCNF: Every Determinant is a Super Key]
    BCNF --> NF4[4NF: No Multi-Valued Dependencies]
    NF4 --> NF5[5NF: No Join Dependencies]
    style UF fill:#ffcdd2
    style BCNF fill:#fff9c4
    style NF5 fill:#c8e6c9
```

---

## 4. Strong vs Weak Entities

### Strong Entity

An entity that can be uniquely identified on its own using its own attributes (has a primary key).

- **Example:** `Employee` (identified by `Emp_ID`)

### Weak Entity

An entity that **cannot** be uniquely identified on its own. It depends on a Strong Entity for its existence.

- **Example:** `Dependent` (a family member of an employee). The dependent "Riya" is only meaningful in the context of employee Alice.
- **Identifying Relationship:** The relationship connecting a weak entity to its owning strong entity.
- **Partial Key (Discriminator):** The attribute within the weak entity that distinguishes weak entity instances for the same strong entity. (e.g., `Dependent_Name` within the same `Emp_ID`).

---

```
TEXT
Strong Entity         Identifying Rel.     Weak Entity
+----------+          +----------+          +------------+
| EMPLOYEE |=====1====| Has      |====M=====| DEPENDENT  |
+----------+          +----------+          +------------+
| Emp_ID◆  |                                 | Dep_Name~  |
| Name     |                                 | DOB        |
+----------+                                 +------------+
◆ = Primary Key    ~ = Partial Key (Discriminator)
```

> *[!NOTE]*
>
> ***Interview Note:*** *Weak entities are represented with a* ***double rectangle*** *in ER diagrams. The identifying relationship is a* ***double diamond****.*

---

## 5. Participation Constraints (Total vs Partial)

Participation constraints define whether ALL instances of an entity must participate in a relationship.

| Type | Meaning | Notation | Example |
|---|---|---|---|
| Total Participation | Every instance MUST participate | Double line == | Every Employee MUST work in a Departmen |
| Partial Participation | Not every instance needs to participate | Single line — | Not every Employee MANAGES a Departmer |

```
TEXT
EMPLOYEE ===(works_in)=== DEPARTMENT
         total              partial
(Every employee must       (A department can
 work in a department)      exist without a manager)
```

---

## 6. ER to Relational Schema Conversion (Full Walkthrough)

Converting an ER diagram to actual SQL tables is a core interview skill.

### Rules:

1. **Each Strong Entity** → becomes a Table. Its attributes → columns. PK → Primary Key.
2. **Each Weak Entity** → becomes a Table. Its Partial Key + Owner's PK = Composite Primary Key.
3. **1:1 Relationship** → Add FK of one entity into the other (prefer the total participation side).
4. **1:M Relationship** → Add FK of the '1' side into the 'M' side's table.
5. **M:N Relationship** → Create a **new Bridge/Junction Table** with FKs from both entities as the composite PK.
6. **Attributes of a Relationship** → Go into the Bridge Table.

### Example: University ER → SQL

**ER Diagram:**

- `Student (S_ID, Name)` enrolls in `Course (C_ID, Title)` — M:N relationship with attribute `Grade`.

**Conversion:**

```
SQL
-- Rule 1: Strong entities become tables
CREATE TABLE Student (
    S_ID INT PRIMARY KEY,
    Name VARCHAR(50)
);

CREATE TABLE Course (
    C_ID   INT PRIMARY KEY,
    Title  VARCHAR(100)
);

-- Rule 5: M:N → Bridge Table
-- Rule 6: Relationship attribute (Grade) goes into bridge table
CREATE TABLE Enrollment (
    S_ID    INT,
    C_ID    INT,
    Grade   CHAR(1),
    PRIMARY KEY (S_ID, C_ID),
    FOREIGN KEY (S_ID) REFERENCES Student(S_ID),
    FOREIGN KEY (C_ID) REFERENCES Course(C_ID)
);
```

---

## 7. Denormalization

**Denormalization** is the process of intentionally introducing redundancy into a normalized database to **improve read performance**.

> *[!NOTE]*
>
> ***When to Denormalize:*** *In **OLAP** (Online Analytical Processing) / Data Warehousing systems where read performance is critical and data rarely changes. JOINs across many normalized tables are expensive for analytics.*

**Example:** Instead of joining `Employee`, `Department`, and `Location` tables every time for a report, store `Dept_Name` directly in the `Employee` table.

| Approach | Use Case | Trade-off |
|---|---|---|
| Normalization | OLTP (Online Transaction Processing) — frequent INSERT/UPDATE/DELETE | Eliminates redundancy, slower reads due to JOINs |
| Denormalization | OLAP (Data Warehousing, Reports, Analytics) | Faster reads, introduces redundancy, harder to maintain |

> *[!WARNING]*
>
> ***Interview Trap:*** *Never say "Denormalization is bad." It is a valid and often necessary design choice for analytical systems. Star Schema and Snowflake Schema in data warehouses are intentionally denormalized.*

---

# ■ CHAPTER END REVISIONS ■

## ■ 5-Minute Quick Revision

1. **ER Model:** Visual representation (Entity, Attribute, Relationship).
2. **Anomalies:** Insertion, Update, and Deletion errors caused by unnormalized data.
3. **1NF:** No multi-valued attributes (atomic values only).
4. **2NF:** No partial dependencies (non-prime relying on part of a composite PK).
5. **3NF:** No transitive dependencies (non-prime relying on another non-prime).
6. **BCNF:** Stricter 3NF. For every `X -> Y`, `X` must be a super key.

## ■ Common Mistakes Students Make in Interviews

1. **Confusing 2NF and 3NF:** 2NF deals with **Partial** dependencies (requires a composite primary key to happen). 3NF deals with **Transitive** dependencies (A -> B -> C).
2. **Forgetting BCNF:** Many students stop at 3NF. Remember that BCNF removes anomalies where a non-prime attribute determines a prime attribute.
3. **Decomposing without linking:** When you decompose a table in an interview, ALWAYS explicitly mention what the Foreign Key will be in the new tables to connect them back.

## ■ Top 5 Placement MCQs

**Q1. Which normal form dictates that all attributes must be single-valued (atomic)?**

A) 1NF

B) 2NF

C) 3NF

D) BCNF

***Answer: A) 1NF.***

**Q2. A table is in 2NF if it is in 1NF and:**

A) Has no transitive dependencies.

B) Has no partial dependencies.

C) Has no foreign keys.

D) Every determinant is a candidate key.

***Answer: B) Has no partial dependencies.***

**Q3. If A -> B and B -> C, then A -> C. This is called:**

A) Trivial dependency

B) Partial dependency

C) Transitive dependency

D) Multi-valued dependency

***Answer: C) Transitive dependency.***

**Q4. A table has fields (EmpID, ProjectID, EmpName, ProjectName). The Primary key is (EmpID, ProjectID). What is the highest normal form of this table?**

A) 1NF

B) 2NF

C) 3NF

D) Unnormalized

***Answer: A) 1NF.***

*This table violates 2NF because of **two** partial dependencies:*

- `EmpID → EmpName` *(EmpName depends only on part of the composite PK)*

- `ProjectID → ProjectName` *(ProjectName depends only on part of the composite PK)*

*Since non-prime attributes depend on only a part of the composite Primary Key* `{EmpID, ProjectID}`*, the table has partial dependencies and therefore **cannot be in 2NF**. Its highest achieved normal form is **1NF** (assuming all attributes are atomic).*

**Q5. For a relation in BCNF, every functional dependency X -> Y implies that:**

A) Y is a prime attribute

B) X is a super key

C) Y is a super key

D) X is a foreign key

***Answer: B) X is a super key.***

## ■ Top 5 Interview Questions

1. **Explain the 3 types of Data Anomalies with examples.**
   • *Answer:* Mention Insertion (can't insert because PK is null), Update (updating one fact requires updating many rows), Deletion (deleting one row deletes unintended facts). Give the Student-Course examples.
2. **What is the difference between 3NF and BCNF?**
   • *Answer:* In 3NF, if X -> Y, either X is a super key OR Y is a prime attribute. BCNF removes the second exception. In BCNF, X *must* be a super key. BCNF is stricter.
3. **Can a table be in 3NF but not in 2NF?**
   • *Answer:* No. Normalization is sequential. To be in 3NF, a table MUST first be in 2NF, which means it MUST first be in 1NF.
4. **What is Denormalization and when would you use it?**
   • *Answer:* Denormalization is intentionally adding redundancy back into a database to speed up read queries (avoiding expensive JOINs). Used heavily in Data Warehousing (OLAP).
5. **Draw an ER diagram for a Library Management System.**
   • *Answer:* Show Entities (Book, Member, Librarian), Attributes (Book_ID, Title, Member_ID), and Relationships (Member *Borrows* Book - M:N relationship requiring a bridge table in implementation).

# ■■■■■ Chapter 3: The Complete SQL Handbook

Every SQL topic needed for TCS, Infosys, Wipro, Amazon, Microsoft, and Oracle placements — with examples, outputs, MCQs, and interview answers.

## 1. Sample Database Setup

> *[!NOTE]*
>
> *All queries in this chapter use these two tables. Run this first.*

```
SQL
CREATE TABLE Departments (
    Dept_ID   INT PRIMARY KEY,
    Dept_Name VARCHAR(50),
    Location  VARCHAR(50)
);

INSERT INTO Departments VALUES (1, 'HR',      'Mumbai');
INSERT INTO Departments VALUES (2, 'IT',      'Bangalore');
INSERT INTO Departments VALUES (3, 'Finance', 'Delhi');
INSERT INTO Departments VALUES (4, 'Sales',   'Chennai');

CREATE TABLE Employees (
    Emp_ID     INT PRIMARY KEY,
    Emp_Name   VARCHAR(50),
    Salary     DECIMAL(10,2),
    Dept_ID    INT,
    Manager_ID INT
);

INSERT INTO Employees VALUES (101, 'Alice',   90000, 1, NULL);
INSERT INTO Employees VALUES (102, 'Bob',     80000, 2, 101);
INSERT INTO Employees VALUES (103, 'Charlie', 75000, 2, 101);
INSERT INTO Employees VALUES (104, 'David',   50000, 3, 102);
INSERT INTO Employees VALUES (105, 'Eve',     80000, 2, 101);
INSERT INTO Employees VALUES (106, 'Frank',   60000, 4, 102);
INSERT INTO Employees VALUES (107, 'Grace',   60000, 4, 102);
INSERT INTO Employees VALUES (108, 'Henry',   NULL,  3, 101);
```

**Employees Table:**

| Emp_ID | Emp_Name | Salary | Dept_ID | Manager_ID |
|---|---|---|---|---|
| 101 | Alice | 90000 | 1 | NULL |
| 102 | Bob | 80000 | 2 | 101 |
| 103 | Charlie | 75000 | 2 | 101 |
| 104 | David | 50000 | 3 | 102 |
| 105 | Eve | 80000 | 2 | 101 |
| 106 | Frank | 60000 | 4 | 102 |

| Emp_ID | Emp_Name | Salary | Dept_ID | Manager_ID |
|---|---|---|---|---|
| 107 | Grace | 60000 | 4 | 102 |
| 108 | Henry | NULL | 3 | 101 |

---

## 2. SQL Data Types

Choosing the right data type is essential for storage efficiency and data integrity.

| Category | Data Type | Description | Example |
|---|---|---|---|
| Numeric | INT | Whole number, 4 bytes | Age, Emp_ID |
| | FLOAT | Approximate decimal, 4 bytes | GPS coordinates |
| | DECIMAL(p,s) | Exact decimal, p total digits, s after dot | Salary, Price |
| String | CHAR(n) | Fixed-length string, always uses n bytes | Gender 'M'/'F' |
| | VARCHAR(n) | Variable-length string, max n bytes | Names, Emails |
| | TEXT | Large text, no size limit | Blog content |
| Date/Time | DATE | Date only: YYYY-MM-DD | Birth_Date |
| | TIME | Time only: HH:MM:SS | Shift start time |
| | DATETIME | Date + Time (no timezone) | Order_Timestamp |
| | TIMESTAMP | Date + Time (auto timezone conversion) | Last login |
| Boolean | BOOLEAN | TRUE (1) or FALSE (0) | is_active |

> *[!TIP]*
>
> ***CHAR vs VARCHAR:*** *Use* `CHAR` *when all values are the same length (like a 2-letter country code). Use* `VARCHAR` *for variable-length data (like names) — it saves space by not padding with blanks.*
>
> ***DATETIME vs TIMESTAMP:*** `TIMESTAMP` *is stored in UTC and converts to the session timezone.* `DATETIME` *has no timezone awareness. Use* `TIMESTAMP` *for "when did this happen" records.*

---

## 3. Basic SELECT Queries

### SELECT

```sql
SELECT Emp_Name, Salary FROM Employees;
```

Fetches only `Emp_Name` and `Salary` columns. `SELECT *` fetches all columns (avoid in production).

### WHERE

```sql
SELECT * FROM Employees WHERE Salary > 70000;
```

**Output:** Alice (90000), Bob (80000), Charlie (75000), Eve (80000).

### DISTINCT

Removes duplicate values from results.

```sql
SELECT DISTINCT Dept_ID FROM Employees;
```

**Output:** 1, 2, 3, 4 (each department ID appears only once, even though IT has 3 employees).

### ORDER BY

```sql
SELECT Emp_Name, Salary FROM Employees ORDER BY Salary DESC;
```

`DESC` = highest first. `ASC` = lowest first (default).

**Output:** Alice (90000) → Bob (80000) → Eve (80000) → Charlie (75000) → Frank (60000) → Grace (60000) → David (50000) → Henry (NULL).

> *[!TIP]*
>
> ***Interview Note:*** *NULL values appear LAST in ascending ORDER BY and FIRST in descending ORDER BY (standard SQL behavior).*

### LIMIT / TOP / FETCH

```sql
-- MySQL / PostgreSQL
SELECT * FROM Employees ORDER BY Salary DESC LIMIT 3;

-- SQL Server
SELECT TOP 3 * FROM Employees ORDER BY Salary DESC;

-- Oracle (12c+)
SELECT * FROM Employees ORDER BY Salary DESC FETCH FIRST 3 ROWS ONLY;
```

**Output:** Top 3 — Alice (90000), Bob (80000), Eve (80000).

### OFFSET

Skips a number of rows before returning results. Used with LIMIT for **pagination**.

```sql
-- Skip the first 3 rows, return the next 3 (Page 2 of results)
SELECT * FROM Employees ORDER BY Salary DESC LIMIT 3 OFFSET 3;
-- Output: Charlie (75000), Frank (60000), Grace (60000)

-- Shorthand (MySQL): LIMIT offset, count
SELECT * FROM Employees ORDER BY Salary DESC LIMIT 3, 3;
```

> *[!TIP]*
>
> ***Pagination Formula:*** `OFFSET = (page_number - 1) × page_size`
>
> *Page 1: LIMIT 10 OFFSET 0 | Page 2: LIMIT 10 OFFSET 10 | Page 3: LIMIT 10 OFFSET 20*

---

## 4. Logical & Filter Operators

### AND, OR, NOT

```sql
-- AND: Both conditions must be true
SELECT * FROM Employees WHERE Dept_ID = 2 AND Salary > 75000;
-- Output: Bob (80000), Eve (80000)

-- OR: Either condition must be true
SELECT * FROM Employees WHERE Dept_ID = 1 OR Dept_ID = 4;
-- Output: Alice, Frank, Grace

-- NOT: Inverts the condition
SELECT * FROM Employees WHERE NOT Dept_ID = 2;
-- Output: Alice, David, Frank, Grace, Henry
```

### IN

Tests if a value matches any value in a list.

```sql
SELECT * FROM Employees WHERE Dept_ID IN (1, 4);
```

**Output:** Alice (Dept 1), Frank (Dept 4), Grace (Dept 4).

> *Equivalent to:* `WHERE Dept_ID = 1 OR Dept_ID = 4` *— but cleaner.*

### NOT IN

Excludes rows that match any value in a list.

```sql
SELECT * FROM Employees WHERE Dept_ID NOT IN (1, 4);
-- Output: Bob, Charlie, Eve (Dept 2), David, Henry (Dept 3)
```

> *[!WARNING]*
>
> ***NOT IN and NULLs are dangerous!*** *If the list contains even one NULL,* `NOT IN` *returns* ***no rows*** *because SQL cannot confirm something is "not equal to NULL". Use* `NOT EXISTS` *or* `LEFT JOIN ... WHERE IS NULL` *as safer alternatives.*

## BETWEEN

Tests if a value lies within a range (inclusive on both ends).

```sql
SQL
SELECT * FROM Employees WHERE Salary BETWEEN 60000 AND 80000;
```

**Output:** Bob (80000), Charlie (75000), Eve (80000), Frank (60000), Grace (60000).

## LIKE

Pattern matching for strings.

| Pattern | Meaning |
|---|---|
| 'A%' | Starts with A |
| '%e' | Ends with e |
| '%li%' | Contains "li" anywhere |
| 'lie' | Any char, then "li", then any char, then "e" |

```sql
SQL
SELECT * FROM Employees WHERE Emp_Name LIKE 'A%';
-- Output: Alice

SELECT * FROM Employees WHERE Emp_Name LIKE '%e';
-- Output: Alice, Charlie, Grace
```

## IS NULL / IS NOT NULL

> *[!IMPORTANT]*
>
> *You CANNOT use* `= NULL`*. NULL is not a value; it is the absence of a value. You must use* `IS NULL`*.*

```sql
SQL
-- Find employees with no salary recorded
SELECT * FROM Employees WHERE Salary IS NULL;
-- Output: Henry

-- Find employees who have a salary
SELECT * FROM Employees WHERE Salary IS NOT NULL;
-- Output: All except Henry
```

## 5. NULL Handling: COALESCE, IFNULL, NULLIF

### COALESCE

`COALESCE(value1, value2, ...)` returns the **first non-NULL** value in the list. Works in all SQL databases.

```
SQL
SELECT Emp_Name, COALESCE(Salary, 0) AS Salary
FROM Employees;
```

**Output:** Henry's salary shows as 0 instead of NULL. All others unchanged.

> *[!TIP]*
>
> ***Memory Trick:*** *COALESCE = "Use this, or if null, use that."*

### IFNULL (MySQL specific)

`IFNULL(value, replacement)` — returns `replacement` only if `value` is NULL. Simpler than COALESCE for 2-argument cases.

```
SQL
SELECT Emp_Name, IFNULL(Salary, 0) AS Salary FROM Employees;
-- Same output as COALESCE example above. Henry shows 0.
```

> *[!NOTE]*
>
> `IFNULL` *is MySQL-specific.* `COALESCE` *is the ANSI SQL standard and works in all databases (MySQL, PostgreSQL, SQL Server, Oracle).*

### NULLIF

`NULLIF(value1, value2)` — returns NULL if both values are **equal**; otherwise returns `value1`. Useful to avoid divide-by-zero errors.

```
SQL
-- Avoid divide-by-zero: if denominator is 0, return NULL instead of error
SELECT Emp_Name, Salary / NULLIF(Dept_ID, 0) AS Result
FROM Employees;

-- Practical: Treat empty string '' the same as NULL
SELECT NULLIF(Emp_Name, '') AS Clean_Name FROM Employees;
```

---

## 6. Aggregate Functions

Aggregate functions operate on a **set of rows** and return a **single value**.

| Function | What it does |
|---|---|
| COUNT(*) | Counts all rows including NULLs |
| COUNT(col) | Counts non-NULL values in that column |
| SUM(col) | Sum of all non-NULL values |
| AVG(col) | Average of all non-NULL values |
| MIN(col) | Minimum value |
| MAX(col) | Maximum value |

```sql
SQL
SELECT
    COUNT(*)         AS Total_Rows,      -- 8
    COUNT(Salary)    AS Salaried_Emps,   -- 7 (Henry excluded)
    SUM(Salary)      AS Total_Payroll,   -- 495000
    AVG(Salary)      AS Avg_Salary,      -- 70714.28
    MIN(Salary)      AS Min_Salary,      -- 50000
    MAX(Salary)      AS Max_Salary       -- 90000
FROM Employees;
```

> *[!WARNING]*
>
> `AVG()` *ignores NULLs. Alice's NULL salary is NOT counted in the denominator, so AVG(Salary) = 495000 / 7, not 495000 / 8.*

---

## 7. GROUP BY & HAVING

### GROUP BY

Groups rows that have the same values and allows aggregate functions per group.

```sql
SQL
SELECT Dept_ID, COUNT(*) AS Headcount, AVG(Salary) AS Avg_Salary
FROM Employees
GROUP BY Dept_ID;
```

**Output:**

| Dept_ID | Headcount | Avg_Salary |
|---|---|---|
| 1 | 1 | 90000 |
| 2 | 3 | 78333.33 |
| 3 | 2 | 50000 |
| 4 | 2 | 60000 |

### HAVING

Filters **groups** after aggregation. `WHERE` cannot be used with aggregate functions.

```
SQL
-- Departments with more than 1 employee
SELECT Dept_ID, COUNT(*) AS Headcount
FROM Employees
GROUP BY Dept_ID
HAVING COUNT(*) > 1;
```

**Output:** Dept 2 (3 employees), Dept 3 (2), Dept 4 (2).

> *[!TIP]*
>
> ***Memory Trick:***
>
> `WHERE` = *Filter **rows** (before GROUP BY)*
>
> `HAVING` = *Filter **groups** (after GROUP BY)*

### Multiple Column Grouping

You can GROUP BY two or more columns to create more granular groups.

```
SQL
-- Group by Department AND Manager to see headcount per manager within each dept
SELECT Dept_ID, Manager_ID, COUNT(*) AS Headcount, AVG(Salary) AS Avg_Salary
FROM Employees
WHERE Salary IS NOT NULL
GROUP BY Dept_ID, Manager_ID;
```

**Output:** Each unique (Dept_ID, Manager_ID) combination becomes its own group. This is useful for analyzing data at a finer level of detail than single-column grouping.

---

## 8. Constraints

Constraints enforce rules on data in a table.

| Constraint | Purpose |
|---|---|
| PRIMARY KEY | Unique + NOT NULL. Uniquely identifies each row. |
| FOREIGN KEY | Links to Primary Key of another table. Enforces referential integrity. |
| UNIQUE | No duplicate values allowed. Unlike PK, can have one NULL. |
| NOT NULL | Column cannot have NULL values. |
| CHECK | Validates a condition before inserting/updating. |
| DEFAULT | Assigns a default value if none is provided. |

```sql
CREATE TABLE Orders (
    Order_ID   INT PRIMARY KEY,
    Order_Date DATE DEFAULT GETDATE(),
    Amount     DECIMAL(10,2) CHECK (Amount > 0),
    Status     VARCHAR(20) NOT NULL,
    Emp_ID     INT,
    FOREIGN KEY (Emp_ID) REFERENCES Employees(Emp_ID)
);
```

### AUTO_INCREMENT / IDENTITY

Automatically generates a unique integer for each new row. Used for surrogate primary keys.

```sql
-- MySQL: AUTO_INCREMENT
CREATE TABLE Products (
    Product_ID INT AUTO_INCREMENT PRIMARY KEY,
    Name       VARCHAR(100) NOT NULL
);
INSERT INTO Products (Name) VALUES ('Laptop');  -- Product_ID = 1 (auto)
INSERT INTO Products (Name) VALUES ('Mouse');   -- Product_ID = 2 (auto)

-- SQL Server: IDENTITY(seed, increment)
CREATE TABLE Products (
    Product_ID INT IDENTITY(1,1) PRIMARY KEY,  -- Start at 1, increment by 1
    Name       VARCHAR(100) NOT NULL
);

-- PostgreSQL: SERIAL or GENERATED ALWAYS AS IDENTITY
CREATE TABLE Products (
    Product_ID SERIAL PRIMARY KEY,
    Name       VARCHAR(100) NOT NULL
);
```

> *[!TIP]*
>
> *To reset the auto-increment counter in MySQL:* `ALTER TABLE Products AUTO_INCREMENT = 1;`

---

## 9. DDL: CREATE, ALTER, DROP, TRUNCATE

### CREATE DATABASE

```sql
CREATE DATABASE CompanyDB;

-- Create only if it doesn't exist (safe re-run)
CREATE DATABASE IF NOT EXISTS CompanyDB;

-- Use the database
USE CompanyDB;
```

## CREATE TABLE

```sql
-- Full syntax with all constraint types
CREATE TABLE Employees (
    Emp_ID     INT            AUTO_INCREMENT PRIMARY KEY,
    Emp_Name   VARCHAR(50)    NOT NULL,
    Email      VARCHAR(100)   UNIQUE,
    Salary     DECIMAL(10,2)  DEFAULT 30000.00 CHECK (Salary >= 0),
    Dept_ID    INT,
    Hire_Date  DATE           DEFAULT (CURDATE()),
    FOREIGN KEY (Dept_ID) REFERENCES Departments(Dept_ID) ON DELETE SET NULL
);

-- Create table from another table's structure + data
CREATE TABLE Employees_Backup AS SELECT * FROM Employees;

-- Create table structure only (no data)
CREATE TABLE Employees_Empty AS SELECT * FROM Employees WHERE 1=0;
```

## ALTER

Modifies the structure of an existing table.

```sql
-- Add a new column
ALTER TABLE Employees ADD Email VARCHAR(100);

-- Modify column data type
ALTER TABLE Employees MODIFY Emp_Name VARCHAR(100);          -- MySQL
ALTER TABLE Employees ALTER COLUMN Emp_Name VARCHAR(100);    -- SQL Server

-- Rename a column (MySQL 8.0+)
ALTER TABLE Employees RENAME COLUMN Emp_Name TO Full_Name;

-- Drop a column
ALTER TABLE Employees DROP COLUMN Email;

-- Rename the table
RENAME TABLE Employees TO Staff;        -- MySQL
EXEC sp_rename 'Employees', 'Staff';   -- SQL Server
```

## DROP vs TRUNCATE

```sql
-- Remove ALL rows, keep structure (DDL, fast, no rollback in MySQL)
TRUNCATE TABLE Employees;

-- Permanently destroy the entire table (structure + data)
DROP TABLE Employees;

-- Safe drop (no error if table doesn't exist)
DROP TABLE IF EXISTS Employees;
```

See the full comparison table in Chapter 1.

## 10. DML: UPDATE & DELETE

### UPDATE

```sql
-- Give IT department a 10% salary raise
UPDATE Employees
SET Salary = Salary * 1.10
WHERE Dept_ID = 2;
```

> *[!WARNING]*
>
> *Always use a `WHERE` clause with UPDATE. Without it, **every row** in the table gets updated.*

### DELETE

```sql
-- Remove a specific employee
DELETE FROM Employees WHERE Emp_ID = 108;

-- Remove all employees from Finance dept
DELETE FROM Employees WHERE Dept_ID = 3;
```

> *[!WARNING]*
>
> *Always use a `WHERE` clause with DELETE. Without it, **every row** is deleted (same as TRUNCATE but slower).*

## 11. JOINS (Complete Guide)

```text
         [ SQL JOINS ]
        /      |      \
   [Inner]  [Outer]  [Cross] [Self]
          /   |   \
      [Left][Right][Full]
```

### INNER JOIN

Returns only rows with a match in **both** tables.

```sql
SELECT E.Emp_Name, D.Dept_Name
FROM Employees E
INNER JOIN Departments D ON E.Dept_ID = D.Dept_ID;
```

**Output:** 8 rows. All employees matched to their department name.

*(Henry in Dept 3=Finance also appears. Sales dept has Frank and Grace.)*

### LEFT JOIN

All rows from left table; NULLs for unmatched right-side columns.

```
SQL
SELECT E.Emp_Name, D.Dept_Name
FROM Employees E
LEFT JOIN Departments D ON E.Dept_ID = D.Dept_ID;
```

**Output:** All 8 employees. If any employee had a non-existent Dept_ID, their Dept_Name would be NULL.

### RIGHT JOIN

All rows from right table; NULLs for unmatched left-side columns.

```
SQL
SELECT E.Emp_Name, D.Dept_Name
FROM Employees E
RIGHT JOIN Departments D ON E.Dept_ID = D.Dept_ID;
```

**Output:** All 4 departments. If a department had no employees, Emp_Name would be NULL.

### FULL OUTER JOIN

All rows from both tables. NULLs where no match exists on either side.

```
SQL
-- SQL Server / PostgreSQL
SELECT E.Emp_Name, D.Dept_Name
FROM Employees E
FULL OUTER JOIN Departments D ON E.Dept_ID = D.Dept_ID;

-- MySQL workaround (MySQL lacks FULL OUTER JOIN)
SELECT E.Emp_Name, D.Dept_Name FROM Employees E LEFT JOIN Departments D ON E.Dept_ID = D.Dept_ID
UNION
SELECT E.Emp_Name, D.Dept_Name FROM Employees E RIGHT JOIN Departments D ON E.Dept_ID = D.Dept_ID;
```

### CROSS JOIN

Returns the **Cartesian product** — every row of Table A paired with every row of Table B.

```
SQL
SELECT E.Emp_Name, D.Dept_Name
FROM Employees E
CROSS JOIN Departments D;
```

**Output:** 8 employees × 4 departments = **32 rows**.

*Used rarely. Practical use: generating all combinations (e.g., all products × all stores).*

### SELF JOIN ■■■■■

A table joined with **itself**. Classic use: finding an employee's manager (who is also an employee).

```
SQL
SELECT E.Emp_Name AS Employee, M.Emp_Name AS Manager
FROM Employees E
LEFT JOIN Employees M ON E.Manager_ID = M.Emp_ID;
```

**Output:**

| Employee | Manager |
|---|---|
| Alice | NULL |
| Bob | Alice |
| Charlie | Alice |
| David | Bob |
| Eve | Alice |
| Frank | Bob |
| Grace | Bob |
| Henry | Alice |

---

## 12. Set Operations

Set operations combine results of two or more SELECT queries. Both queries must have the same number of columns and compatible data types.

### UNION

Combines results and **removes duplicates**.

```
SQL
SELECT Dept_ID FROM Employees
UNION
SELECT Dept_ID FROM Departments;
-- Output: 1, 2, 3, 4 (unique values only)
```

### UNION ALL

Combines results and **keeps duplicates**. Faster than UNION.

```sql
SELECT Dept_ID FROM Employees
UNION ALL
SELECT Dept_ID FROM Departments;
-- Output: 1,2,2,2,3,3,4,4 from Employees + 1,2,3,4 from Departments = 12 rows
```

### INTERSECT

Returns rows that appear in **both** result sets.

```sql
SELECT Dept_ID FROM Employees
INTERSECT
SELECT Dept_ID FROM Departments;
-- Output: 1, 2, 3, 4 (all Dept_IDs in Employees also exist in Departments)
```

> *[!NOTE]*
>
> *MySQL supports INTERSECT only from version 8.0.31+. SQL Server and PostgreSQL support it natively.*

### EXCEPT / MINUS

Returns rows from the **first** query that do **not** appear in the second.

```sql
-- SQL Server / PostgreSQL
SELECT Dept_ID FROM Departments
EXCEPT
SELECT Dept_ID FROM Employees;
-- Output: Empty (all departments have at least one employee in our data)

-- Oracle uses MINUS instead of EXCEPT
SELECT Dept_ID FROM Departments
MINUS
SELECT Dept_ID FROM Employees;
```

---

## 13. Subqueries: EXISTS, ANY, ALL

### EXISTS

Returns TRUE if the subquery returns at least one row.

```sql
-- Find departments that have at least one employee
SELECT Dept_Name FROM Departments D
WHERE EXISTS (
    SELECT 1 FROM Employees E WHERE E.Dept_ID = D.Dept_ID
);
-- Output: HR, IT, Finance, Sales (all departments have employees in our data)
```

---

### ANY

Returns TRUE if the comparison is true for **at least one** value in the subquery result.

```sql
SQL
-- Employees earning more than at least one Finance dept employee
SELECT Emp_Name FROM Employees
WHERE Salary > ANY (SELECT Salary FROM Employees WHERE Dept_ID = 3);
-- (Finance salaries: 50000, NULL) — any employee earning > 50000 qualifies
-- Output: Alice, Bob, Charlie, Eve, Frank, Grace
```

### ALL

Returns TRUE if the comparison is true for **all** values in the subquery result.

```sql
SQL
-- Employees earning more than ALL Finance dept employees (ignores NULLs)
SELECT Emp_Name FROM Employees
WHERE Salary > ALL (SELECT Salary FROM Employees WHERE Dept_ID = 3 AND Salary IS NOT NULL);
-- Finance non-null salaries: 50000 only. Employees earning > 50000:
-- Output: Alice, Bob, Charlie, Eve, Frank, Grace
```

---

## 14. CASE Expression

CASE is SQL's IF-THEN-ELSE. Returns a value based on conditions.

```sql
SQL
SELECT Emp_Name, Salary,
    CASE
        WHEN Salary >= 80000 THEN 'High'
        WHEN Salary >= 60000 THEN 'Medium'
        WHEN Salary IS NULL  THEN 'Not Set'
        ELSE 'Low'
    END AS Salary_Grade
FROM Employees;
```

**Output:**

| Emp_Name | Salary | Salary_Grade |
|---|---|---|
| Alice | 90000 | High |
| Bob | 80000 | High |
| Charlie | 75000 | Medium |
| David | 50000 | Low |
| Eve | 80000 | High |
| Frank | 60000 | Medium |

| Emp_Name | Salary | Salary_Grade |
|---|---|---|
| Grace | 60000 | Medium |
| Henry | NULL | Not Set |

---

## 15. Views

A **View** is a virtual table based on a stored SELECT query. It does not store data physically.

```
SQL
-- Create a view
CREATE VIEW IT_Employees AS
SELECT Emp_Name, Salary
FROM Employees
WHERE Dept_ID = 2;

-- Use it like a table
SELECT * FROM IT_Employees;
-- Output: Bob, Charlie, Eve

-- Drop a view
DROP VIEW IT_Employees;
```

**Why use Views?**

- **Security:** Expose only specific columns/rows to certain users.
- **Simplicity:** Hide complex JOIN logic behind a simple view name.
- **Consistency:** Ensure all queries use the same logic.

> *[!IMPORTANT]*
>
> *A view is NOT a physical copy of data. Every time you query a view, the underlying SELECT runs again.*

### UPDATE VIEW (CREATE OR REPLACE)

Modify an existing view's definition without dropping it first.

```
SQL
-- Update the view to also include Dept_ID
CREATE OR REPLACE VIEW IT_Employees AS
SELECT Emp_Name, Salary, Dept_ID
FROM Employees
WHERE Dept_ID = 2;

-- SQL Server equivalent
ALTER VIEW IT_Employees AS
SELECT Emp_Name, Salary, Dept_ID
FROM Employees
WHERE Dept_ID = 2;
```

> *[!NOTE]*

---

> *A **simple view** (based on a single table, no GROUP BY/DISTINCT/aggregate) is **updatable** — you can run INSERT/UPDATE/DELETE on it and the underlying table is modified. Complex views are read-only.*

### Materialized View

A **Materialized View** physically stores the result of the query on disk (unlike a regular view which re-executes on every query).

| Feature | Regular View | Materialized View |
|---|---|---|
| Storage | No physical storage | Physically stored on disk |
| Performance | Re-executes query each time | Reads from stored snapshot (very fast) |
| Data freshness | Always current | Can be stale (needs REFRESH) |
| Use case | Simple abstraction/security | Expensive aggregations, reporting |

```
SQL
-- PostgreSQL syntax (MySQL does not support Materialized Views natively)
CREATE MATERIALIZED VIEW dept_salary_summary AS
SELECT Dept_ID, COUNT(*) AS Headcount, AVG(Salary) AS Avg_Salary
FROM Employees
GROUP BY Dept_ID;

-- Refresh the data manually when needed
REFRESH MATERIALIZED VIEW dept_salary_summary;
```

> *[!TIP]*
>
> ***Interview Answer:** "Materialized Views are used in data warehousing and reporting where query results are large and complex but data doesn't need to be real-time. They trade freshness for speed."*

---

## 16. Indexes

An **Index** is a database object that speeds up data retrieval at the cost of extra storage and slower writes.

```
SQL
-- Create an index on Salary for fast salary-based lookups
CREATE INDEX idx_salary ON Employees(Salary);

-- Create a unique index (no duplicate values allowed)
CREATE UNIQUE INDEX idx_emp_name ON Employees(Emp_Name);

-- Drop an index
DROP INDEX idx_salary ON Employees;  -- MySQL
DROP INDEX idx_salary;               -- Oracle / PostgreSQL
```

| Type | Description |
|---|---|
| Clustered Index | Determines physical sort order of data. ONE per table. Usually the Primary Ke |

---

| Type | Description |
| --- | --- |
| Non-Clustered Index | Separate structure with pointers to actual rows. Many per table allowed. |

---

## 17. Window Functions ■■■■■

Window functions perform calculations across a **window** (set of related rows) without collapsing results like `GROUP BY`.

**Syntax:**

```
SQL
function_name() OVER (PARTITION BY col ORDER BY col)
```

### ROW_NUMBER()

Assigns a unique sequential integer to each row. No ties — always unique.

```
SQL
SELECT Emp_Name, Salary,
    ROW_NUMBER() OVER (ORDER BY Salary DESC) AS Row_Num
FROM Employees WHERE Salary IS NOT NULL;
```

**Output:** Alice=1, Bob=2, Eve=3, Charlie=4, Frank=5, Grace=6, David=7

### RANK()

Assigns the same rank to tied values. **Skips** next rank numbers after a tie.

```
SQL
SELECT Emp_Name, Salary,
    RANK() OVER (ORDER BY Salary DESC) AS Rnk
FROM Employees WHERE Salary IS NOT NULL;
```

**Output:** Alice=1, Bob=2, Eve=2, Charlie=4 (skips 3), Frank=5, Grace=5, David=7

### DENSE_RANK()

Assigns the same rank to tied values. Does **NOT** skip next rank numbers.

```
SQL
SELECT Emp_Name, Salary,
    DENSE_RANK() OVER (ORDER BY Salary DESC) AS Dense_Rnk
FROM Employees WHERE Salary IS NOT NULL;
```

**Output:** Alice=1, Bob=2, Eve=2, Charlie=3, Frank=4, Grace=4, David=5

> *[!TIP]*

---

> *### ■ Memory Trick: ROW_NUMBER vs RANK vs DENSE_RANK*
>
> *Imagine 2 people tie for 2nd place (80000 salary — Bob and Eve):*
>
> ** `ROW_NUMBER()`: 1, **2, 3**, 4 — Always unique, no ties*
>
> ** `RANK()`: 1, **2, 2**, 4 — Ties get same rank, SKIPS 3*
>
> ** `DENSE_RANK()`: 1, **2, 2**, 3 — Ties get same rank, NO skip*

### PARTITION BY

Divides the result set into groups (partitions) and applies the window function within each group independently.

```
SQL
-- Rank employees BY SALARY within each department
SELECT Emp_Name, Dept_ID, Salary,
    DENSE_RANK() OVER (PARTITION BY Dept_ID ORDER BY Salary DESC) AS Dept_Rank
FROM Employees WHERE Salary IS NOT NULL;
```

**Output (key rows):**

| Emp_Name | Dept_ID | Salary | Dept_Rank |
|---|---|---|---|
| Alice | 1 | 90000 | 1 |
| Bob | 2 | 80000 | 1 |
| Eve | 2 | 80000 | 1 |
| Charlie | 2 | 75000 | 2 |
| David | 3 | 50000 | 1 |
| Frank | 4 | 60000 | 1 |
| Grace | 4 | 60000 | 1 |

---

## 18. ■■■■■ Classic Placement Query Patterns

These are the most-asked SQL coding questions in every placement interview.

### 1. 2nd Highest Salary

```sql
-- Method 1: Subquery (works everywhere)
SELECT MAX(Salary) AS Second_Highest
FROM Employees
WHERE Salary < (SELECT MAX(Salary) FROM Employees);
-- Output: 80000

-- Method 2: DENSE_RANK (best for N-th highest)
WITH Ranked AS (
    SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employees WHERE Salary IS NOT NULL
)
SELECT Salary FROM Ranked WHERE rnk = 2;
-- Output: 80000
```

## 2. Nth Highest Salary (Generic)

```sql
-- Replace N with any number (e.g., 3 for 3rd highest)
WITH Ranked AS (
    SELECT Emp_Name, Salary,
           DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employees WHERE Salary IS NOT NULL
)
SELECT Emp_Name, Salary FROM Ranked WHERE rnk = 3;
-- For N=3: Output: Charlie (75000)
```

## 3. Find Duplicate Records

```sql
-- Find salaries that appear more than once
SELECT Salary, COUNT(*) AS Count
FROM Employees
WHERE Salary IS NOT NULL
GROUP BY Salary
HAVING COUNT(*) > 1;
-- Output: 80000 (Bob and Eve), 60000 (Frank and Grace)
```

## 4. Remove Duplicate Records (Keep one)

```sql
-- Keep the row with the lowest Emp_ID, delete the rest
WITH CTE AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY Emp_Name ORDER BY Emp_ID) AS rn
    FROM Employees
)
DELETE FROM CTE WHERE rn > 1;
```

## 5. Department-Wise Highest Salary

```sql
-- Method 1: GROUP BY + MAX
SELECT D.Dept_Name, MAX(E.Salary) AS Max_Salary
FROM Employees E
JOIN Departments D ON E.Dept_ID = D.Dept_ID
GROUP BY D.Dept_Name;

-- Method 2: PARTITION BY (shows employee name too)
WITH Ranked AS (
    SELECT Emp_Name, Dept_ID, Salary,
           RANK() OVER (PARTITION BY Dept_ID ORDER BY Salary DESC) AS rnk
    FROM Employees WHERE Salary IS NOT NULL
)
SELECT Emp_Name, Dept_ID, Salary FROM Ranked WHERE rnk = 1;
```

**Output:**

| Dept_Name | Max_Salary |
|---|---|
| HR | 90000 |
| IT | 80000 |
| Finance | 50000 |
| Sales | 60000 |

## 6. Employees Earning More Than Their Manager

```sql
SELECT E.Emp_Name AS Employee, E.Salary AS Emp_Salary,
       M.Emp_Name AS Manager,  M.Salary AS Mgr_Salary
FROM Employees E
JOIN Employees M ON E.Manager_ID = M.Emp_ID
WHERE E.Salary > M.Salary;
-- Output: Empty (no employee earns more than their manager in our dataset)
-- Test it: UPDATE Employees SET Salary = 95000 WHERE Emp_ID = 102;
-- Then Bob (95000) > Alice (90000) → Bob appears in result
```

## 7. Top 3 Salaries (Distinct)

```sql
WITH Ranked AS (
    SELECT DISTINCT Salary,
           DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employees WHERE Salary IS NOT NULL
)
SELECT Salary FROM Ranked WHERE rnk <= 3;
-- Output: 90000, 80000, 75000
```

## 8. Consecutive Records with Same Salary (Using LAG)

```sql
-- Find employees where their salary equals the previous employee's salary
SELECT Emp_Name, Salary,
       LAG(Salary) OVER (ORDER BY Emp_ID) AS Prev_Salary
FROM Employees
WHERE Salary = LAG(Salary) OVER (ORDER BY Emp_ID);
```

> *[!NOTE]*
>
> `LAG(col)` *accesses the value from the* ***previous row****.* `LEAD(col)` *accesses the* ***next row****.*

## 19. String Functions (MySQL)

String functions are essential for data cleaning and transformation. Highly tested in coding rounds.

| Function | Description | Example | Output |
|---|---|---|---|
| UPPER(str) | Converts to uppercase | UPPER('alice') | ALICE |
| LOWER(str) | Converts to lowercase | LOWER('ALICE') | alice |
| LENGTH(str) | Returns byte length | LENGTH('Alice') | 5 |
| CHAR_LENGTH(str) | Returns character count | CHAR_LENGTH('Alice') | 5 |
| SUBSTRING(str, pos, len) | Extracts substring | SUBSTRING('Alice', 2, 3) | lic |
| LEFT(str, n) | First N characters | LEFT('Alice', 3) | Ali |
| RIGHT(str, n) | Last N characters | RIGHT('Alice', 3) | ice |
| TRIM(str) | Removes leading/trailing spaces | TRIM('  hi  ') | hi |
| LTRIM(str) | Removes leading spaces | LTRIM('  hi') | hi |
| RTRIM(str) | Removes trailing spaces | RTRIM('hi  ') | hi |
| REPLACE(str, from, to) | Replaces substring | REPLACE('Hello World', 'World', 'SQL') | Hello SQL |
| CONCAT(s1, s2, ...) | Joins strings | CONCAT('Alice', ' - ', 'HR') | Alice - HR |
| INSTR(str, substr) | Position of first occurrence | INSTR('Alice', 'li') | 2 |
| LPAD(str, len, pad) | Left-pads to length | LPAD('5', 4, '0') | 0005 |
| RPAD(str, len, pad) | Right-pads to length | RPAD('5', 4, '0') | 5000 |
| REVERSE(str) | Reverses the string | REVERSE('SQL') | LQS |

```sql
-- Practical Examples
SELECT UPPER(Emp_Name), CHAR_LENGTH(Emp_Name) AS Name_Length
FROM Employees;

-- Extract first name from 'Alice Smith'
SELECT SUBSTRING_INDEX('Alice Smith', ' ', 1) AS First_Name;
-- Output: Alice

-- Pad employee ID to 5 digits: 101 → 00101
SELECT LPAD(Emp_ID, 5, '0') AS Padded_ID FROM Employees;
```

## 20. Date and Time Functions (MySQL)

| Function | Description | Example | Output |
|---|---|---|---|
| NOW() | Current date and time | NOW() | 2025-01-15 14:30:00 |
| CURDATE() | Current date only | CURDATE() | 2025-01-15 |
| CURTIME() | Current time only | CURTIME() | 14:30:00 |
| DATE(datetime) | Extracts date part | DATE(NOW()) | 2025-01-15 |
| YEAR(date) | Extracts year | YEAR('2025-01-15') | 2025 |
| MONTH(date) | Extracts month | MONTH('2025-01-15') | 1 |
| DAY(date) | Extracts day | DAY('2025-01-15') | 15 |
| DATEDIFF(d1, d2) | Days between two dates | DATEDIFF('2025-01-15', '2025-01-01') | 14 |
| DATE_ADD(date, INTERVAL n unit) | Adds to a date | DATE_ADD('2025-01-01', INTERVAL 30 DAY) | 2025-01-31 |
| DATE_SUB(date, INTERVAL n unit) | Subtracts from a date | DATE_SUB('2025-01-31', INTERVAL 1 MONTH) | 2024-12-31 |
| DATE_FORMAT(date, format) | Formats a date | DATE_FORMAT('2025-01-15', '%d-%b-%Y') | 15-Jan-2025 |
| TIMESTAMPDIFF(unit, d1, d2) | Difference in specified unit | TIMESTAMPDIFF(YEAR, '2000-01-01', '2025-01-01') | 25 |
| EXTRACT(unit FROM date) | Extracts a date part | EXTRACT(YEAR FROM '2025-01-15') | 2025 |

```sql
-- Employees hired more than 5 years ago (assuming a Hire_Date column)
SELECT Emp_Name FROM Employees
WHERE TIMESTAMPDIFF(YEAR, Hire_Date, CURDATE()) > 5;

-- Format today's date
SELECT DATE_FORMAT(CURDATE(), '%d/%m/%Y') AS Formatted_Date;
-- Output: 15/01/2025
```

## 21. Mathematical Functions

| Function | Description | Example | Output |
|---|---|---|---|
| ROUND(n, d) | Rounds to d decimal places | ROUND(456.567, 2) | 456.57 |
| CEIL(n) / CEILING(n) | Rounds UP to nearest integer | CEIL(4.1) | 5 |
| FLOOR(n) | Rounds DOWN to nearest integer | FLOOR(4.9) | 4 |
| ABS(n) | Absolute value | ABS(-50) | 50 |
| MOD(n, d) | Remainder (modulo) | MOD(10, 3) | 1 |
| POWER(n, p) | n raised to power p | POWER(2, 10) | 1024 |
| SQRT(n) | Square root | SQRT(144) | 12 |
| TRUNCATE(n, d) | Truncates to d decimal places (no rounding) | TRUNCATE(456.567, 2) | 456.56 |

```sql
SQL
-- Round average salary to 2 decimal places
SELECT ROUND(AVG(Salary), 2) AS Avg_Salary FROM Employees;

-- Find employees with odd Emp_IDs
SELECT Emp_Name FROM Employees WHERE MOD(Emp_ID, 2) = 1;
```

---

## 22. Transactions: BEGIN, COMMIT, ROLLBACK, SAVEPOINT

A **Transaction** is a sequence of SQL statements treated as a single unit of work.

```sql
SQL
-- Basic Transaction
START TRANSACTION;  -- or BEGIN;

    UPDATE Employees SET Salary = Salary - 5000 WHERE Emp_ID = 101;
    UPDATE Employees SET Salary = Salary + 5000 WHERE Emp_ID = 102;

COMMIT;  -- Make changes permanent

-- If something goes wrong:
ROLLBACK;  -- Undo all changes since START TRANSACTION
```

### SAVEPOINT — Partial Rollback

A SAVEPOINT lets you rollback to a specific point within a transaction without undoing everything.

```sql
START TRANSACTION;

    INSERT INTO Employees VALUES (110, 'Zara', 70000, 1, 101);
    SAVEPOINT after_insert;  -- Mark this point

    UPDATE Employees SET Salary = -1 WHERE Emp_ID = 110;  -- Mistake!

ROLLBACK TO SAVEPOINT after_insert;  -- Undo only the UPDATE, keep the INSERT

COMMIT;  -- Now only the INSERT is committed
```

> *[!TIP]*
>
> ***Memory Trick:***
>
> - `COMMIT` = *Save progress in a game*
>
> - `ROLLBACK` = *Restart from last save*
>
> - `SAVEPOINT` = *Create a checkpoint mid-game*

---

## 23. CTEs (Common Table Expressions) — WITH Clause

A **CTE** is a named temporary result set that exists only for the duration of a single query. It makes complex queries more readable.

### Basic CTE

```sql
-- Without CTE (hard to read nested subquery)
SELECT * FROM (
    SELECT Emp_Name, Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employees WHERE Salary IS NOT NULL
) AS ranked WHERE rnk = 2;

-- With CTE (clean and readable)
WITH RankedSalaries AS (
    SELECT Emp_Name, Salary,
           DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employees WHERE Salary IS NOT NULL
)
SELECT Emp_Name, Salary FROM RankedSalaries WHERE rnk = 2;
-- Output: Bob (80000), Eve (80000)
```

### Multiple CTEs

```
SQL
WITH
    IT_Employees AS (
        SELECT * FROM Employees WHERE Dept_ID = 2
    ),
    IT_Stats AS (
        SELECT AVG(Salary) AS Avg_IT_Salary FROM IT_Employees
    )
SELECT E.Emp_Name, E.Salary, S.Avg_IT_Salary
FROM IT_Employees E, IT_Stats S;
```

## ■■■■■ Recursive CTE (Very Important for Interviews!)

A **Recursive CTE** calls itself and is used for hierarchical data (org charts, category trees, folder structures).

**Use Case:** Find all employees under a manager (the entire org hierarchy).

```
SQL
-- Find Alice's entire reporting chain (all employees reporting to Alice, directly or indirectly)
WITH RECURSIVE OrgChart AS (
    -- Anchor member: Start with Alice (Emp_ID = 101)
    SELECT Emp_ID, Emp_Name, Manager_ID, 0 AS Level
    FROM Employees
    WHERE Emp_ID = 101

    UNION ALL

    -- Recursive member: Find employees whose manager is already in the CTE
    SELECT E.Emp_ID, E.Emp_Name, E.Manager_ID, O.Level + 1
    FROM Employees E
    INNER JOIN OrgChart O ON E.Manager_ID = O.Emp_ID
)
SELECT Emp_ID, Emp_Name, Level FROM OrgChart;
```

**Output:**

| Emp_ID | Emp_Name | Level |
|---|---|---|
| 101 | Alice | 0 (top) |
| 102 | Bob | 1 |
| 103 | Charlie | 1 |
| 105 | Eve | 1 |
| 108 | Henry | 1 |
| 104 | David | 2 |
| 106 | Frank | 2 |
| 107 | Grace | 2 |

> *[!IMPORTANT]*

> ***Recursive CTE Structure:***
>
> *1. **Anchor Member:** The base case (starting row).*
>
> *2. `UNION ALL`*
>
> *3. **Recursive Member:** References the CTE itself to get the next level.*
>
> *MySQL supports Recursive CTEs from version 8.0+.*

---

## 24. Stored Procedures

A **Stored Procedure** is a precompiled, reusable block of SQL stored in the database. Call it with `CALL`.

```
SQL
-- Create a stored procedure to give a department a salary raise
DELIMITER $$

CREATE PROCEDURE GiveSalaryRaise(
    IN dept_id INT,         -- IN: input parameter (read-only inside proc)
    IN raise_pct DECIMAL(5,2)  -- percentage raise
)
BEGIN
    UPDATE Employees
    SET Salary = Salary * (1 + raise_pct / 100)
    WHERE Dept_ID = dept_id AND Salary IS NOT NULL;

    SELECT CONCAT('Salary updated for Dept_ID: ', dept_id) AS Message;
END$$

DELIMITER ;

-- Call the procedure
CALL GiveSalaryRaise(2, 10);  -- Give IT department a 10% raise
```

### IN vs OUT vs INOUT Parameters

| Parameter Type | Description |
|---|---|
| IN | Read-only input value. Changes inside procedure do NOT affect the caller. |
| OUT | Output value. Procedure sets it; caller can read it after the call. |
| INOUT | Both input and output. Caller passes a value in; procedure can modify it. |

```sql
-- Procedure with OUT parameter
DELIMITER $$
CREATE PROCEDURE GetDeptHeadcount(
    IN dept_id INT,
    OUT headcount INT
)
BEGIN
    SELECT COUNT(*) INTO headcount
    FROM Employees
    WHERE Dept_ID = dept_id;
END$$
DELIMITER ;

-- Calling with OUT
CALL GetDeptHeadcount(2, @count);
SELECT @count;  -- Output: 3
```

> *[!TIP]*
>
> ***Interview Answer:*** *Stored Procedures improve performance (precompiled), reduce network traffic (only CALL sent, not full SQL), and improve security (users call procedures without direct table access).*

---

## 25. User-Defined Functions (UDF)

A **Function** is similar to a Stored Procedure but MUST return a single value and can be used inside a SELECT statement.

```sql
DELIMITER $$

CREATE FUNCTION GetSalaryGrade(salary DECIMAL(10,2))
RETURNS VARCHAR(10)
DETERMINISTIC   -- Same input always produces same output
BEGIN
    DECLARE grade VARCHAR(10);
    IF salary >= 80000 THEN SET grade = 'High';
    ELSEIF salary >= 60000 THEN SET grade = 'Medium';
    ELSEIF salary IS NULL THEN SET grade = 'Not Set';
    ELSE SET grade = 'Low';
    END IF;
    RETURN grade;
END$$

DELIMITER ;

-- Use in SELECT (like a built-in function)
SELECT Emp_Name, Salary, GetSalaryGrade(Salary) AS Grade
FROM Employees;
```

| Feature | Stored Procedure | Function |
|---|---|---|
| Returns value | Optional (can return 0 or many values) | Must return exactly 1 value |
| Used in SELECT | ■ Cannot be called in SELECT | ■ Can be used in SELECT |

---

| Feature | Stored Procedure | Function |
|---|---|---|
| Called with | CALL proc_name() | Used in expression: SELECT func_name() |
| Transaction control | Can use COMMIT/ROLLBACK | Cannot |

---

## 26. Triggers

A **Trigger** is a special stored procedure that automatically fires in response to a DML event (`INSERT`, `UPDATE`, `DELETE`) on a table.

**Timing:** `BEFORE` (fires before the DML) or `AFTER` (fires after the DML).

```
SQL
-- AFTER INSERT Trigger: Log every new employee into an audit table
CREATE TABLE Audit_Log (
    Log_ID     INT AUTO_INCREMENT PRIMARY KEY,
    Action     VARCHAR(50),
    Emp_ID     INT,
    Log_Time   DATETIME DEFAULT NOW()
);

DELIMITER $$

CREATE TRIGGER after_employee_insert
AFTER INSERT ON Employees
FOR EACH ROW
BEGIN
    INSERT INTO Audit_Log(Action, Emp_ID)
    VALUES ('INSERT', NEW.Emp_ID);
END$$

-- BEFORE UPDATE Trigger: Prevent salary from being set below 0
CREATE TRIGGER before_salary_update
BEFORE UPDATE ON Employees
FOR EACH ROW
BEGIN
    IF NEW.Salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative!';
    END IF;
END$$

DELIMITER ;
```

> *[!NOTE]*
>
> ***`NEW` and `OLD` in Triggers:***
>
> *- `NEW.column` = The new value being inserted/updated.*
>
> *- `OLD.column` = The old value before the update/delete.*
>
> *- `INSERT` triggers: only `NEW` is available.*
>
> *- `DELETE` triggers: only `OLD` is available.*

---

- `UPDATE` *triggers: both* `NEW` *and* `OLD` *are available.*

---

## 27. More Window Functions: NTILE, FIRST_VALUE, LAST_VALUE

### NTILE(n)

Divides rows into `n` equal buckets and assigns a bucket number to each row.

```sql
SQL
-- Divide employees into 4 salary quartiles
SELECT Emp_Name, Salary,
    NTILE(4) OVER (ORDER BY Salary DESC) AS Quartile
FROM Employees WHERE Salary IS NOT NULL;
```

**Output:**

| Emp_Name | Salary | Quartile |
|---|---|---|
| Alice | 90000 | 1 (top 25%) |
| Bob | 80000 | 1 |
| Eve | 80000 | 2 |
| Charlie | 75000 | 2 |
| Frank | 60000 | 3 |
| Grace | 60000 | 3 |
| David | 50000 | 4 (bottom 25%) |

### FIRST_VALUE() and LAST_VALUE()

```sql
SQL
-- Show each employee's salary alongside the highest salary in their department
SELECT Emp_Name, Dept_ID, Salary,
    FIRST_VALUE(Salary) OVER (PARTITION BY Dept_ID ORDER BY Salary DESC) AS Dept_Max_Salary,
    LAST_VALUE(Salary)  OVER (PARTITION BY Dept_ID ORDER BY Salary DESC
                              ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS Dept_Min_Salary
FROM Employees WHERE Salary IS NOT NULL;
```

*[!WARNING]*

`LAST_VALUE` *requires* `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` *to see the actual last value in the window. Without this, it only looks at rows up to the current row.*

---

## 28. ROLLUP and CUBE (Subtotals and Grand Totals)

### GROUP BY WITH ROLLUP

Adds subtotals and a grand total row automatically.

```
SQL
SELECT Dept_ID, COUNT(*) AS Headcount, SUM(Salary) AS Total_Salary
FROM Employees
WHERE Salary IS NOT NULL
GROUP BY Dept_ID WITH ROLLUP;
```

**Output:**

| Dept_ID | Headcount | Total_Salary |
|---|---|---|
| 1 | 1 | 90000 |
| 2 | 3 | 235000 |
| 3 | 1 | 50000 |
| 4 | 2 | 120000 |
| NULL | 7 | 495000 ← Grand Total |

> *[!NOTE]*
>
> *The `NULL` in the last row represents the grand total (all groups combined). Use `COALESCE(Dept_ID, 'ALL')` to show "ALL" instead of NULL.*

---

## 29. EXPLAIN — Query Optimization

`EXPLAIN` shows how MySQL executes a query — essential for performance tuning interviews.

```
SQL
EXPLAIN SELECT * FROM Employees WHERE Dept_ID = 2;
```

**Key columns in EXPLAIN output:**

| Column | What to Look For |
|---|---|
| type | ALL = Full scan (bad for large tables). ref/range/const = Index used (good). |
| rows | Estimated number of rows examined. Lower is better. |
| key | The index actually used (NULL means no index used). |
| Extra | Using filesort = Sorting in memory (can be slow). Using index = Index covers t |

```sql
-- Without index: type=ALL, key=NULL (full scan)
EXPLAIN SELECT * FROM Employees WHERE Dept_ID = 2;

-- After creating index:
CREATE INDEX idx_dept ON Employees(Dept_ID);

-- With index: type=ref, key=idx_dept (efficient!)
EXPLAIN SELECT * FROM Employees WHERE Dept_ID = 2;
```

---

## 30. Security: Users, Roles, GRANT & REVOKE (DCL)

Database security controls WHO can access WHAT data and perform WHICH operations. This is managed with **DCL (Data Control Language)**.

### Creating Users

```sql
-- MySQL: Create a new database user
CREATE USER 'john'@'localhost' IDENTIFIED BY 'SecurePass123!';
CREATE USER 'app_user'@'%' IDENTIFIED BY 'AppPass!';
-- '%' = can connect from any host; 'localhost' = only from local machine
```

### GRANT — Give Permissions

```sql
-- Grant specific privileges on a specific table
GRANT SELECT, INSERT ON CompanyDB.Employees TO 'john'@'localhost';

-- Grant all privileges on an entire database
GRANT ALL PRIVILEGES ON CompanyDB.* TO 'app_user'@'%';

-- Grant SELECT on all databases (read-only analyst user)
GRANT SELECT ON *.* TO 'analyst'@'localhost';

-- Allow the user to pass their privileges to others (WITH GRANT OPTION)
GRANT SELECT ON CompanyDB.* TO 'john'@'localhost' WITH GRANT OPTION;

-- Apply changes immediately
FLUSH PRIVILEGES;
```

### REVOKE — Remove Permissions

```sql
-- Remove specific privileges
REVOKE INSERT ON CompanyDB.Employees FROM 'john'@'localhost';

-- Remove all privileges
REVOKE ALL PRIVILEGES ON CompanyDB.* FROM 'john'@'localhost';

-- Drop the user entirely
DROP USER 'john'@'localhost';
```

### Roles (MySQL 8.0+ / PostgreSQL)

Roles are **named groups of privileges**. Instead of granting permissions to each user individually, assign a role.

```sql
-- Create roles
CREATE ROLE 'read_only_role';
CREATE ROLE 'developer_role';

-- Assign privileges to the role
GRANT SELECT ON CompanyDB.* TO 'read_only_role';
GRANT SELECT, INSERT, UPDATE, DELETE ON CompanyDB.* TO 'developer_role';

-- Assign the role to a user
GRANT 'read_only_role' TO 'john'@'localhost';
GRANT 'developer_role' TO 'jane'@'localhost';

-- Activate the role for the session
SET DEFAULT ROLE 'read_only_role' TO 'john'@'localhost';
```

**Privilege Levels (from broadest to narrowest):**

| Privilege Level | Syntax | Scope |
|---|---|---|
| Global | ON . | All databases |
| Database | ON db_name.* | All tables in one database |
| Table | ON dbname.tablename | One specific table |
| Column | ON db_name.table(col) | One specific column |

> *[!IMPORTANT]*
>
> ***Interview Answer:*** *"DCL (Data Control Language) manages database access permissions.* `GRANT` *gives privileges and* `REVOKE` *removes them. Roles simplify permission management by grouping privileges that can be assigned to multiple users at once."*

---

## 31. More Classic Placement Query Patterns

### 9. Running Total (Cumulative Sum)

```
SQL
SELECT Emp_Name, Salary,
    SUM(Salary) OVER (ORDER BY Emp_ID) AS Running_Total
FROM Employees WHERE Salary IS NOT NULL;
```

## 10. Moving Average (3-row window)

```
SQL
SELECT Emp_Name, Salary,
    AVG(Salary) OVER (ORDER BY Emp_ID ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS Moving_Avg
FROM Employees WHERE Salary IS NOT NULL;
```

## 11. Employees Without a Department (Orphan Records)

```
SQL
-- Using LEFT JOIN — more efficient than NOT IN with NULLs
SELECT E.Emp_Name
FROM Employees E
LEFT JOIN Departments D ON E.Dept_ID = D.Dept_ID
WHERE D.Dept_ID IS NULL;
```

## 12. Departments With No Employees

```
SQL
SELECT D.Dept_Name
FROM Departments D
LEFT JOIN Employees E ON D.Dept_ID = E.Dept_ID
WHERE E.Emp_ID IS NULL;
```

## 13. Find All Managers (Employees Who Manage Others)

```
SQL
SELECT DISTINCT M.Emp_Name AS Manager
FROM Employees E
JOIN Employees M ON E.Manager_ID = M.Emp_ID;
-- Output: Alice, Bob
```

## 14. Employees Who Are NOT Managers

```
SQL
SELECT Emp_Name FROM Employees
WHERE Emp_ID NOT IN (
    SELECT DISTINCT Manager_ID FROM Employees WHERE Manager_ID IS NOT NULL
);
-- Output: Charlie, David, Eve, Frank, Grace, Henry
```

### 15. Salary Greater Than Department Average

```sql
SELECT E.Emp_Name, E.Salary, D.Avg_Dept_Salary
FROM Employees E
JOIN (
    SELECT Dept_ID, AVG(Salary) AS Avg_Dept_Salary
    FROM Employees GROUP BY Dept_ID
) D ON E.Dept_ID = D.Dept_ID
WHERE E.Salary > D.Avg_Dept_Salary;
```

### 16. Pivot Table: Count employees per department in columns

```sql
SELECT
    SUM(CASE WHEN Dept_ID = 1 THEN 1 ELSE 0 END) AS HR_Count,
    SUM(CASE WHEN Dept_ID = 2 THEN 1 ELSE 0 END) AS IT_Count,
    SUM(CASE WHEN Dept_ID = 3 THEN 1 ELSE 0 END) AS Finance_Count,
    SUM(CASE WHEN Dept_ID = 4 THEN 1 ELSE 0 END) AS Sales_Count
FROM Employees;
```

### 18. Delete Duplicate Emails (Classic LeetCode #196)

```sql
-- Keep the row with the smallest ID, delete the rest
DELETE E1 FROM Employees E1
INNER JOIN Employees E2
WHERE E1.Emp_ID > E2.Emp_ID AND E1.Emp_Name = E2.Emp_Name;
```

---

## ■ 5-Minute Quick Revision

1. **SELECT / WHERE / DISTINCT / ORDER BY / LIMIT** — basic retrieval and filtering.
2. **AND, OR, NOT / IN / BETWEEN / LIKE** — logical and pattern filters.
3. **IS NULL / IS NOT NULL / COALESCE** — NULL handling. Never use `= NULL`.
4. **Aggregate Functions** — COUNT/SUM/AVG/MIN/MAX ignore NULLs (except COUNT(*)).
5. **GROUP BY** — groups rows. **HAVING** — filters groups. Never use aggregate in WHERE.
6. **Constraints** — PK (unique+notnull), FK (referential), UNIQUE, NOT NULL, CHECK, DEFAULT.
7. **JOINS** — INNER (both match), LEFT (all left), RIGHT (all right), FULL (all both), CROSS (Cartesian), SELF (same table).
8. **UNION** (dedup) vs **UNION ALL** (keep all). INTERSECT (common). EXCEPT/MINUS (difference).
9. **EXISTS** (any row?), **ANY** (any match?), **ALL** (all match?).
10. **CASE** — SQL if-then-else. **COALESCE** — first non-NULL.
11. **VIEW** — virtual table, no physical storage.
12. **INDEX** — speeds up SELECT, slows INSERT/UPDATE/DELETE.

13. **ROW_NUMBER** (always unique) vs **RANK** (ties + gaps) vs **DENSE_RANK** (ties, no gaps).
14. **PARTITION BY** — apply window function per group independently.

## ■ Common Mistakes in Interviews

1. **Using `= NULL` instead of `IS NULL`** — SQL will always return empty result with `= NULL`.
2. **Using aggregate in WHERE** — Use HAVING for aggregates, WHERE for row-level filters.
3. **RANK vs DENSE_RANK confusion** — Remember RANK skips numbers, DENSE_RANK does not.
4. **Forgetting SELF JOIN needs aliases** — `FROM Employees E JOIN Employees M` — both must have different aliases.
5. **UNION removes duplicates, UNION ALL keeps them** — UNION is slower due to dedup step.
6. **FULL OUTER JOIN not supported in MySQL natively** — simulate with LEFT UNION RIGHT.

## ■ Top 10 Placement MCQs

**Q1. Which clause is used to filter results AFTER aggregation?**

A) WHERE B) HAVING C) GROUP BY D) ORDER BY

> ***Answer: B) HAVING.***

**Q2. What does `SELECT COUNT(*) FROM Employees` return if Henry has NULL salary?**

A) 7 B) 8 C) 0 D) NULL

> ***Answer: B) 8.*** *COUNT(*) counts ALL rows including NULLs.*

**Q3. CROSS JOIN between 5 rows and 4 rows produces how many rows?**

A) 9 B) 20 C) 5 D) 4

> ***Answer: B) 20.*** *Cartesian product: 5 × 4 = 20.*

**Q4. Which window function does NOT produce gaps after ties?**

A) RANK() B) ROW_NUMBER() C) DENSE_RANK() D) NTILE()

> ***Answer: C) DENSE_RANK().***

**Q5. Which of the following is used to check if a subquery returns any row?**

A) ANY B) ALL C) EXISTS D) IN

> ***Answer: C) EXISTS.***

**Q6. What is the correct way to check for NULL in SQL?**

A) `= NULL` B) `IS NULL` C) `== NULL` D) `EQUALS NULL`

> ***Answer: B) IS NULL.***

**Q7. UNION vs UNION ALL — which is faster?**

A) UNION B) UNION ALL C) Both are equal D) Depends on the table

> ***Answer: B) UNION ALL.*** *No duplicate removal step.*

**Q8. A view in SQL is:**

A) A copy of data stored on disk

B) A virtual table based on a SELECT query

C) A stored procedure

D) An index on a table

***Answer: B) A virtual table based on a SELECT query.***

**Q9. Which function returns the value of the previous row's column?**

A) LEAD() B) LAG() C) FIRST_VALUE() D) RANK()

***Answer: B) LAG().***

**Q10. Which SQL set operator removes duplicates by default?**

A) UNION ALL B) INTERSECT C) UNION D) EXCEPT

***Answer: C) UNION.*** *(UNION ALL keeps duplicates; INTERSECT and EXCEPT also deduplicate, but UNION is the standard answer here.)*

## ■ Top 10 Interview Questions

1. **What is the difference between WHERE and HAVING?**
   • WHERE filters individual rows before grouping. HAVING filters groups after GROUP BY. Aggregate functions (like MAX, COUNT) cannot be used in WHERE.

2. **Explain RANK(), DENSE_RANK(), and ROW_NUMBER() with an example.**
   • ROW_NUMBER: always unique (1,2,3). RANK: same for ties, skips next number (1,2,2,4). DENSE_RANK: same for ties, no skip (1,2,2,3).

3. **What is a SELF JOIN? When do you use it?**
   • A self join joins a table to itself using two different aliases. Classic use: finding an employee's manager, who is also an employee in the same table.

4. **What is the difference between UNION and UNION ALL?**
   • UNION combines results from two queries and removes duplicates (slower). UNION ALL combines results and keeps all duplicates (faster).

5. **How do you find the Nth highest salary?**
   • Use DENSE_RANK() OVER (ORDER BY Salary DESC) in a CTE, then filter WHERE rnk = N. DENSE_RANK handles ties correctly.

6. **What is a View? Why use it over a direct query?**
   • A view is a saved SELECT query treated as a virtual table. Use it for security (restrict column access), simplicity (hide complex JOINs), and consistency (same logic reused everywhere).

7. **What is COALESCE? How is it different from ISNULL?**
   • COALESCE(a,b,c,...) returns the first non-NULL value and accepts multiple arguments. It is ANSI SQL standard. ISNULL(a,b) is SQL Server-specific and accepts only two arguments.

8. **What is the difference between DELETE and TRUNCATE?**
   • DELETE is DML, logs each row, supports WHERE, can be rolled back. TRUNCATE is DDL, deallocates pages, no WHERE, cannot be rolled back in MySQL (can be in PostgreSQL inside a transaction).

9. **What is the difference between EXISTS and IN?**

   • EXISTS checks if a subquery returns any row (stops at first match, more efficient for large datasets). IN checks if a value matches any value in a list/subquery result (fetches all values first).

10. **Write a query to find employees who earn more than the average salary.**

```sql
SQL
    SELECT Emp_Name, Salary
    FROM Employees
    WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

# ■■■■■ Chapter 4: Advanced DBMS & Interview Topics

This chapter separates the beginners from the experts. You must master Transactions, ACID properties, and Indexing to crack a product-based company interview.

---

## 1. Transactions & ACID Properties

A **Transaction** is a sequence of one or more database operations (reads and writes) that are treated as a single logical unit of work. Transactions are managed to ensure the ACID properties are maintained.

> *[!NOTE]*
>
> ***Real-World Banking Analogy:***
>
> *Alice wants to transfer $100 to Bob.*
>
> * ***Step 1:*** *Read Alice's Balance ($500)*
>
> * ***Step 2:*** *Deduct $100 from Alice (New Balance: $400)*
>
> * ***Step 3:*** *Read Bob's Balance ($200)*
>
> * ***Step 4:*** *Add $100 to Bob (New Balance: $300)*
>
> *What if the server crashes right after Step 2? Alice lost $100, but Bob never got it! This is why we need ACID properties.*

### ■ ACID Properties (Extremely Important)

• **Atomicity (All or Nothing):** The transaction either completes all 4 steps successfully (`COMMIT`) or fails completely and reverses any changes (`ROLLBACK`). The $100 isn't lost in the void.

• **Consistency:** The database must remain in a valid state. (e.g., The total sum of Alice and Bob's money must remain $700 before and after the transaction).

• **Isolation:** If Alice and Charlie both send money to Bob at the exact same millisecond, the DBMS must isolate the transactions so they don't overwrite each other's updates.

• **Durability:** Once a transaction is `COMMIT`ted, the changes are permanent. Even if the server catches fire 1 second later, the data is safe on the hard drive.

### ■ Transaction Flow Diagram

```
TEXT
                [ BEGIN TRANSACTION ]
                        |
                        v
                [ READ Alice ($500) ]
                        |
                        v
                [ WRITE Alice ($400) ]
                        |
                (Server Crashes here?) ---> [ ROLLBACK ] ---> (Alice back to $500)
                        |
                        v
                [ READ Bob ($200) ]
                        |
                        v
                [ WRITE Bob ($300) ]
                        |
                        v
                    [ COMMIT ]
                        |
                (Data is now DURABLE)
```

## 2. Concurrency Control & Locking

When thousands of users access a database at the same time (Concurrency), we use Locks to prevent data corruption.

### The Locking Mechanism

Imagine a public restroom with one toilet. If someone is inside, they lock the door. Others must wait in a queue.

- **Shared Lock (Read Lock - 'S'):** Multiple people can *read* the data at the same time. (Like multiple people reading a public notice board).
- **Exclusive Lock (Write Lock - 'X'):** Only ONE person can *write/update* the data. Everyone else is locked out until they finish.

### Lock Compatibility Matrix

| Requested ↓ \ Held → | Shared (S) | Exclusive (X) |
|---|---|---|
| Shared (S) | ■ Compatible | ■ Conflict |
| Exclusive (X) | ■ Conflict | ■ Conflict |

## 3. ■■■■■ Concurrency Anomalies (Must Know for Interviews)

These are the problems that occur when multiple transactions run concurrently without proper isolation.

### 1. Dirty Read

**Problem:** Transaction T2 reads data that T1 has modified but **NOT YET COMMITTED**. If T1 then rolls back, T2 has read "dirty" (non-existent) data.

```
TEXT
T1: UPDATE Salary = 90000 (not committed yet)
T2: READ Salary → sees 90000  ← DIRTY READ!
T1: ROLLBACK  (Salary is back to 80000)
T2: Is now working with wrong data!
```

### 2. Non-Repeatable Read

**Problem:** Transaction T1 reads the same row twice, but T2 **commits an UPDATE** between the two reads. T1 gets different values.

```
TEXT
T1: READ Salary → 80000
T2: UPDATE Salary = 90000 → COMMIT
T1: READ Salary again → 90000  ← Different value! Non-Repeatable!
```

### 3. Phantom Read

**Problem:** Transaction T1 executes the same range query twice, but T2 **commits an INSERT or DELETE** between the reads. T1 sees different rows (like ghosts appearing/disappearing).

```
TEXT
T1: SELECT * WHERE Dept=2 → returns 3 rows
T2: INSERT new employee in Dept=2 → COMMIT
T1: SELECT * WHERE Dept=2 → returns 4 rows  ← Phantom row appeared!
```

---

## 4. ■■■■ Transaction Isolation Levels

Isolation levels control how much concurrent transactions can interfere with each other. Higher isolation = more safety, less performance.

| Isolation Level | Dirty Read | Non-Repeatable Read | Phantom Read |
|---|---|---|---|
| Read Uncommitted | ■ Possible | ■ Possible | ■ Possible |
| Read Committed | ■ Prevented | ■ Possible | ■ Possible |
| Repeatable Read | ■ Prevented | ■ Prevented | ■ Possible |
| Serializable | ■ Prevented | ■ Prevented | ■ Prevented |

```mermaid
graph TD
    RU[Read Uncommitted - Lowest Isolation] --> RC[Read Committed]
    RC --> RR[Repeatable Read - MySQL Default]
    RR --> S[Serializable - Highest Isolation]
    style RU fill:#ffcdd2
    style S fill:#c8e6c9
```

```sql
-- Set isolation level for a session (MySQL)
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;  -- MySQL default
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

> *[!NOTE]*
>
> ***MySQL InnoDB default:** Repeatable Read. It prevents dirty reads and non-repeatable reads, and uses **gap locks** to also prevent phantom reads in many cases.*

> *[!TIP]*
>
> ***Interview Answer:** "Serializable is the safest level but has the lowest performance because it essentially runs transactions one at a time. Read Committed is the default in PostgreSQL and many other systems for a balance of safety and performance."*

---

## 5. Two-Phase Locking (2PL) Protocol

**Two-Phase Locking** is the standard protocol to ensure **serializability** (correct concurrent execution).

### The Two Phases:

1. **Growing Phase:** A transaction can ACQUIRE locks but cannot RELEASE any lock.
2. **Shrinking Phase:** A transaction can RELEASE locks but cannot ACQUIRE any new lock.

```mermaid
graph LR
    A[Start] --> B[Growing Phase: Acquire Locks]
    B --> C[Lock Point - Maximum locks held]
    C --> D[Shrinking Phase: Release Locks]
    D --> E[Commit/Rollback]
    style B fill:#fff9c4
    style D fill:#c8e6c9
```

> *[!IMPORTANT]*
>
> ***2PL Guarantees Serializability** but does NOT prevent Deadlocks.*

### Strict 2PL

All Exclusive (Write) locks are held until COMMIT/ROLLBACK. This prevents Cascading Rollbacks (where one transaction's rollback forces other transactions to also rollback).

---

## 6. Schedules and Serializability

A **Schedule** is the interleaved sequence of operations from multiple concurrent transactions.

### Serial Schedule

Transactions execute one after another, with no interleaving. Always correct but slow.

```
TEXT
T1: Read(A), Write(A), Commit
T2: Read(B), Write(B), Commit
```

### Serializable Schedule

A concurrent schedule that produces the same result as *some* serial schedule. This is the goal of concurrency control.

### Conflict Serializability (Tested in Exams!)

Two operations **conflict** if:

1. They belong to different transactions.
2. They operate on the **same data item**.
3. At least one of them is a **WRITE**.

**Precedence Graph (Wait-For Graph) Method:**

- Draw a directed edge T1 → T2 if T1's operation conflicts with T2's and T1 comes first.
- If the graph has **no cycle** → **Schedule is Conflict Serializable**.
- If the graph has **a cycle** → **NOT Conflict Serializable**.

```
TEXT
Example Schedule:
T1: R(A)        W(A)
T2:      W(A)        R(B) W(B)

Conflicts: T1 R(A) vs T2 W(A) → T1 first → Edge T1→T2
           T2 W(A) vs T1 W(A) → T2 first → Edge T2→T1

Graph has cycle T1→T2→T1 → NOT Serializable!
```

---

## 7. ■ Deadlock

A Deadlock occurs when two transactions are waiting for each other to release a lock, resulting in an infinite wait.

**Deadlock Analogy:**

- Transaction A locks the `Employee` table and needs the `Department` table.
- Transaction B locks the `Department` table and needs the `Employee` table.
- Both wait forever. The DBMS must detect this and kill (abort) one of the transactions.

### Deadlock Detection

The DBMS maintains a **Wait-For Graph (WFG)**. A cycle in the WFG indicates a deadlock.

```mermaid
graph LR
    T1["T1 (holds Employee, waits for Department)"] --> T2["T2 (holds Department, waits for Employee)"]
    T2 --> T1
    style T1 fill:#ffcdd2
    style T2 fill:#ffcdd2
```

**Resolution:** DBMS selects a **victim** transaction (usually the one with least work done) and aborts it.

### Deadlock Prevention

- **Wait-Die:** Older transaction waits; younger transaction dies (rolls back).
- **Wound-Wait:** Older transaction wounds (aborts) the younger; younger waits.
- **No-Wait:** Never wait for a lock; if lock is unavailable, immediately rollback.
- **Timeout:** Rollback if lock is not obtained within a set time.

### Deadlock Avoidance

**Banker's Algorithm:** Only grant a lock if the resulting state is "safe" (no deadlock possible). Rarely used in practice due to overhead.

---

## 8. Recovery Techniques

When a database crashes, it must recover to a consistent state.

### Write-Ahead Logging (WAL) ■■■■■

**Rule:** Before any change is written to the actual database on disk, it MUST first be recorded in a **Log File**.

```text
Log Entry format: [Transaction ID, Operation, Old Value, New Value]
Example: [T1, UPDATE, Salary=80000, Salary=90000]

Order of operations:
1. WRITE to LOG (WAL rule — must happen FIRST)
2. Write to Buffer (in memory)
3. COMMIT recorded in log
4. Flush Buffer to disk (physical DB file)
```

**Why WAL?** If the system crashes after step 3 but before step 4, the log can be used to REDO the committed changes and ensure durability.

### REDO and UNDO in Recovery

| Operation | When Used | What it Does |
|---|---|---|
| REDO | Transaction was COMMITTed but changes not yet on disk | Re-apply the committed changes |
| UNDO | Transaction was NOT committed when crash occurred | Reverse the uncommitted changes |

### Checkpointing

A **Checkpoint** is a snapshot of the current DB state written to disk periodically. During recovery, the DBMS only needs to REDO/UNDO transactions that were active AFTER the last checkpoint, drastically reducing recovery time.

```text
Without Checkpoint: Must scan entire log (hours of recovery)
With Checkpoint: Only scan log from last checkpoint (seconds to minutes)
```

---

## 9. Indexing (How Databases Search So Fast)

If you have a table with 10 Million rows, searching for `Emp_ID = 500` would normally require checking every single row one-by-one (a **Full Table Scan**). This is incredibly slow.

> *[!TIP]*
>
> ***The Library Analogy (No Jargon First):***
>
> *Imagine finding a specific recipe in a 1,000-page cookbook that has no index. You have to flip through every single page. (Full Table Scan).*
>
> *Now, imagine the book has an **Index** at the back. You look up "Chicken" in alphabetical order, and it says "Page 450". You jump straight to page 450. (Index Scan).*

### Types of Indexes

| Type | Description | Notes |
|---|---|---|
| Clustered Index | Data rows physically stored in index order. Only one per table. | Usually the Primary Key in MySQL InnoDB |
| Non-Clustered Index | Separate structure with pointers to actual rows. Many per table. | All other indexed columns |
| Dense Index | One index entry per data record | Faster search, more storage |
| Sparse Index | One index entry per block of data | Less storage, slightly slower |
| Composite Index | Index on multiple columns | CREATE INDEX idx ON T(A, B) |
| Partial Index | Index on a subset of rows | PostgreSQL only: WHERE salary > 0 |

| Type | Description | Notes |
|---|---|---|
| Covering Index | All columns needed by query exist in the index itself | No table lookup needed → very fast |

### How Indexes Work Internally (B-Trees and B+ Trees)

Databases use tree data structures to store indexes. The most popular is the **B+ Tree**.

### ■■ B-Tree vs B+ Tree Diagram

```
TEXT
B-Tree: Data can be stored in the middle branches.
      [ 50 (Data) ]
     /             \
 [ 20 (Data) ]  [ 80 (Data) ]

B+ Tree: Data is ONLY stored in the bottom leaves. Leaves are linked!
          [ 50 ]
         /      \
      [ 20 ]   [ 80 ]
        |          |
 [Data: 10,20]->[Data: 50,80]  <-- Linked List at the bottom!
```

**Why B+ Tree is better than B-Tree:**

In a B+ Tree, the leaf nodes are linked together like a train (a Linked List). This makes **Range Queries** (e.g., `WHERE Salary BETWEEN 40000 AND 60000`) blazingly fast because the DBMS just finds 40000 and rides the train sideways!

### Hashing for Indexes

**Static Hashing:** Uses a fixed number of buckets. Problem: if too many records hash to the same bucket (overflow), performance degrades.

**Dynamic (Extendible) Hashing:** The number of buckets grows automatically as the table grows. Used when data size is unpredictable. Avoids the overflow problem.

| Feature | B+ Tree Index | Hash Index |
|---|---|---|
| Equality search = | ■ Fast | ■ Very Fast (O(1)) |
| Range search BETWEEN, >, < | ■ Fast (linked leaves) | ■ Cannot do range queries |
| ORDER BY | ■ Efficient | ■ Not useful |
| Used by default in MySQL | ■ Yes (InnoDB) | ■ No |

---

## 10. Query Processing Pipeline

```
MERMAID
graph TD
    A[SQL Query] --> B[Parser: Syntax Check]
    B --> C[Semantic Analyzer: Table/Column Existence Check]
    C --> D[Query Optimizer: Find Best Execution Plan]
    D --> E[Query Evaluator: Execute the Plan]
    E --> F[Result Set Returned to User]
    style D fill:#fff9c4
```

## Query Optimizer

The optimizer chooses the most efficient execution plan. Two main approaches:

| Type | How it Works |
|---|---|
| Rule-Based Optimization (RBO) | Applies predefined rules (e.g., "push WHERE clause down before JOIN") |
| Cost-Based Optimization (CBO) | Estimates cost (I/O, CPU) of different plans and picks the cheapest one. Uses |

> *[!NOTE]*
>
> *Modern databases use Cost-Based Optimization. MySQL's optimizer can be observed using `EXPLAIN` and `EXPLAIN ANALYZE`.*

## SQL Query Execution Order

This is the actual ORDER SQL clauses are processed internally (different from writing order!):

```
1. FROM / JOIN       ← Get the raw data from tables
2. WHERE             ← Filter rows
3. GROUP BY          ← Group filtered rows
4. HAVING            ← Filter groups
5. SELECT            ← Pick columns / compute expressions
6. DISTINCT          ← Remove duplicates
7. ORDER BY          ← Sort results
8. LIMIT / OFFSET    ← Limit output rows
```

> *[!IMPORTANT]*
>
> ***This is why you CANNOT use a SELECT alias in a WHERE clause** — WHERE is processed BEFORE SELECT!*
>
> *```sql*
>
> *-- WRONG: WHERE sees column before SELECT assigns alias*
>
> *SELECT Salary * 1.1 AS New_Salary FROM Employees WHERE New_Salary > 90000;*
>
> *-- CORRECT: Use HAVING after GROUP BY, or repeat the expression*
>
> *SELECT Salary 1.1 AS New_Salary FROM Employees WHERE Salary 1.1 > 90000;*
>
> *```*

# 11. Advanced SQL Techniques

## Temporary Tables

A **Temporary Table** exists only for the duration of a session or transaction. It is stored in `tempdb` (SQL Server) or in memory/temp storage (MySQL). Useful for storing intermediate results in complex queries.

```
SQL
-- MySQL: Create a temporary table (auto-dropped when session ends)
CREATE TEMPORARY TABLE temp_high_earners AS
SELECT Emp_ID, Emp_Name, Salary
FROM Employees
WHERE Salary > 70000;

-- Use it like a regular table
SELECT * FROM temp_high_earners ORDER BY Salary DESC;

-- Manually drop it (optional — it drops automatically when session ends)
DROP TEMPORARY TABLE temp_high_earners;
```

```
SQL
-- SQL Server: Prefix with # for local temp table (session-scoped)
SELECT Emp_ID, Emp_Name, Salary
INTO #temp_high_earners
FROM Employees
WHERE Salary > 70000;

SELECT * FROM #temp_high_earners;
DROP TABLE #temp_high_earners;

-- ## for global temp tables (visible to all sessions, dropped when all references close)
SELECT * INTO ##global_temp FROM Employees;
```

| Feature | Temporary Table | CTE | Subquery |
|---|---|---|---|
| Persistence | Session/transaction | Single query | Single query |
| Reusable | ■ Yes (multiple queries) | ■ Within same query | ■ No |
| Indexable | ■ Yes | ■ No | ■ No |
| Best for | Large intermediate results, multi-step logic | Readable single queries | Simple inline filters |

> *[!TIP]*
>
> ***Interview Answer:*** *Use Temporary Tables when you need to store a large intermediate result set that is reused multiple times across several queries in the same session. Use CTEs for readability within a single query.*

## Derived Tables (Inline Views)

A **Derived Table** is a subquery used inside a `FROM` clause, treated as a virtual table for that query only. Unlike CTEs, they cannot reference themselves.

```sql
SQL
-- Find departments where the average salary exceeds 70,000
-- The inner SELECT is the Derived Table (aliased as 'dept_avg')
SELECT dept_avg.Dept_ID, dept_avg.Avg_Salary
FROM (
    SELECT Dept_ID, AVG(Salary) AS Avg_Salary
    FROM Employees
    WHERE Salary IS NOT NULL
    GROUP BY Dept_ID
) AS dept_avg                          -- <-- This is the derived table
WHERE dept_avg.Avg_Salary > 70000;

-- Output: Dept 1 (90000), Dept 2 (78333)
```

```sql
SQL
-- Derive the top earner per department, then join back to get their name
SELECT E.Emp_Name, E.Dept_ID, E.Salary
FROM Employees E
JOIN (
    SELECT Dept_ID, MAX(Salary) AS Max_Sal
    FROM Employees
    GROUP BY Dept_ID
) AS dept_max ON E.Dept_ID = dept_max.Dept_ID
            AND E.Salary   = dept_max.Max_Sal;
```

> *[!NOTE]*
>
> ***Derived Table vs CTE:*** *A derived table is embedded directly in the FROM clause. A CTE is defined with* `WITH` *before the query. CTEs are generally preferred for readability; derived tables can be useful for simple one-off transformations.*

---

## Dynamic SQL

**Dynamic SQL** is SQL that is constructed and executed at runtime as a string — useful when table names, column names, or conditions are not known until execution time.

```sql
SQL
-- MySQL: Dynamic SQL using PREPARE and EXECUTE
SET @table_name = 'Employees';
SET @sql = CONCAT('SELECT COUNT(*) FROM ', @table_name);

PREPARE stmt FROM @sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;
-- Output: 8
```

```sql
-- Practical: Dynamic column filter (column name as variable)
SET @col = 'Salary';
SET @val = 80000;
SET @sql = CONCAT('SELECT Emp_Name FROM Employees WHERE ', @col, ' >= ', @val);

PREPARE stmt FROM @sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;
```

```sql
-- SQL Server: Dynamic SQL using sp_executesql (safer — prevents SQL injection)
DECLARE @TableName NVARCHAR(50) = 'Employees';
DECLARE @SQL NVARCHAR(MAX);

SET @SQL = N'SELECT COUNT(*) FROM ' + QUOTENAME(@TableName);
EXEC sp_executesql @SQL;
-- QUOTENAME() wraps in square brackets → [Employees], preventing injection
```

> *[!WARNING]*
>
> ***SQL Injection Risk!*** *Never concatenate raw user input directly into a dynamic SQL string. Always use parameterized queries (`sp_executesql` with parameters in SQL Server, or `PREPARE` with placeholders in MySQL) to prevent SQL injection attacks.*

---

## Partitioning

**Partitioning** splits a large table into smaller, more manageable pieces (partitions) while appearing as one table to the user. Queries that filter on the partition key only scan the relevant partition(s) — called **Partition Pruning**.

### Types of Partitioning

| Type | How it Works | Best For |
|---|---|---|
| RANGE | Rows assigned to partitions based on a value range | Date-based data (orders by year) |
| LIST | Rows assigned based on a discrete list of values | Region/country codes |
| HASH | Rows distributed evenly using a hash function | Load balancing, no natural partition key |
| KEY | Similar to HASH but uses MySQL's own hash function | Default even distribution |

```sql
-- RANGE Partitioning: Split Orders table by year
CREATE TABLE Orders (
    Order_ID   INT NOT NULL,
    Order_Date DATE NOT NULL,
    Amount     DECIMAL(10,2)
)
PARTITION BY RANGE (YEAR(Order_Date)) (
    PARTITION p2022 VALUES LESS THAN (2023),
    PARTITION p2023 VALUES LESS THAN (2024),
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p_future VALUES LESS THAN MAXVALUE
);

-- This query ONLY scans the p2023 partition (Partition Pruning!)
SELECT * FROM Orders WHERE Order_Date BETWEEN '2023-01-01' AND '2023-12-31';
```

```sql
-- LIST Partitioning: Split Employees by region
CREATE TABLE Employees_Regional (
    Emp_ID INT,
    Region VARCHAR(20)
)
PARTITION BY LIST COLUMNS(Region) (
    PARTITION p_north VALUES IN ('Delhi', 'Chandigarh'),
    PARTITION p_south VALUES IN ('Chennai', 'Bangalore'),
    PARTITION p_west  VALUES IN ('Mumbai', 'Pune')
);
```

```sql
-- Manage partitions
ALTER TABLE Orders ADD PARTITION (PARTITION p2025 VALUES LESS THAN (2026));
ALTER TABLE Orders DROP PARTITION p2022;   -- Instantly deletes all 2022 data!
```

**Benefits of Partitioning:**

- ■ **Faster queries** via partition pruning (only relevant partitions scanned)
- ■■ **Fast data archiving** — drop an old partition instead of slow `DELETE`
- ■ **Easier maintenance** — rebuild index on one partition, not the whole table
- ■ **Better I/O distribution** — different partitions on different disks

> *[!IMPORTANT]*
>
> ***Partitioning vs Sharding:***
>
> *- **Partitioning** splits data within a single database instance (one server).*
>
> *- **Sharding** splits data across multiple database instances (multiple servers). Used at massive scale (millions of rows, distributed systems).*

---

# ■ CHAPTER END REVISIONS ■

## ■ 5-Minute Quick Revision

1. **Transaction:** Logical unit of work. BEGIN → operations → COMMIT/ROLLBACK.
2. **ACID:** Atomicity (All/Nothing), Consistency (Valid state), Isolation (No interference), Durability (Permanent).
3. **Concurrency Anomalies:** Dirty Read (uncommitted read), Non-Repeatable Read (same row, different values), Phantom Read (different row count).
4. **Isolation Levels:** Read Uncommitted < Read Committed < Repeatable Read (MySQL default) < Serializable.
5. **Locks:** Shared (Read, multiple allowed), Exclusive (Write, only one).
6. **2PL:** Growing phase (acquire) + Shrinking phase (release). Guarantees serializability.
7. **Schedules:** Serial (always correct), Serializable (equivalent to serial). Detect via Precedence Graph.
8. **Deadlock:** Two transactions waiting infinitely. Detected via Wait-For Graph. Resolved by aborting victim.
9. **WAL (Write-Ahead Logging):** Log written BEFORE data. Enables REDO (committed, not on disk) and UNDO (not committed).
10. **Checkpoint:** Periodic snapshot to reduce recovery time.
11. **Indexing:** Data structure to speed up retrieval. Uses B+ Trees.
12. **B+ Tree:** Stores data only in leaf nodes. Leaf nodes are linked for fast range scans.
13. **Dense Index:** One entry per record. Sparse Index: One entry per block.
14. **Clustered:** ONE per table, physical order. Non-Clustered: MANY per table, separate structure with pointers.
15. **Query Execution Order:** FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.
16. **Temporary Tables:** Session-scoped tables for storing intermediate results. Indexable and reusable within session.
17. **Derived Tables:** Subquery in FROM clause — a one-time inline virtual table.
18. **Dynamic SQL:** SQL built and executed as a string at runtime. Use parameterized queries to prevent SQL injection.
19. **Partitioning:** Splits a large table into smaller partitions. RANGE (dates), LIST (discrete values), HASH (even distribution). Enables partition pruning for fast queries.

## ■ Common Mistakes Students Make in Interviews

1. **Confusing Atomicity and Consistency:** Atomicity guarantees the transaction runs fully or aborts. Consistency guarantees business rules (like money cannot be negative) are upheld before and after.
2. **Thinking Indexes are always good:** Indexes speed up `SELECT` queries, but they slow down `INSERT`, `UPDATE`, and `DELETE` queries because the index tree must be rearranged every time data changes. Don't over-index!
3. **Forgetting Durability:** Durability means the data survives a power loss. It is achieved using database transaction logs (Write-Ahead Logging).
4. **Mixing up Dirty Read and Non-Repeatable Read:** Dirty Read is reading an UNCOMMITTED change. Non-Repeatable Read is reading a COMMITTED change between two reads of the same row.
5. **Saying 2PL prevents Deadlocks:** 2PL guarantees **serializability** but does NOT prevent deadlocks.

6. **Confusing WHERE alias:** You cannot reference a SELECT alias in WHERE because WHERE is processed before SELECT.

## ■ Top 10 Placement MCQs

**Q1. Which ACID property ensures that either all operations execute or none do?**

A) Atomicity B) Consistency C) Isolation D) Durability

***Answer: A) Atomicity.***

**Q2. What type of lock allows multiple transactions to read a data item but not update it?**

A) Exclusive Lock B) Deadlock C) Shared Lock D) Binary Lock

***Answer: C) Shared Lock.***

**Q3. The situation where two transactions wait indefinitely for each other to release a lock is called:**

A) Starvation B) Deadlock C) Concurrency D) Serializability

***Answer: B) Deadlock.***

**Q4. Which data structure is most commonly used for database indexing?**

A) Hash Tables B) Linked Lists C) B+ Trees D) Binary Search Trees

***Answer: C) B+ Trees.***

**Q5. What is a major disadvantage of adding too many indexes to a table?**

A) SELECT queries become slower B) INSERT and UPDATE queries become slower C) Database uses less storage D) Deadlocks occur more frequently

***Answer: B) INSERT and UPDATE queries become slower.***

***Note:*** *DELETE operations also become slower with excessive indexes.*

**Q6. Which isolation level prevents Dirty Reads but NOT Non-Repeatable Reads?**

A) Read Uncommitted B) Read Committed C) Repeatable Read D) Serializable

***Answer: B) Read Committed.***

**Q7. Transaction T2 reads data that T1 has modified but not yet committed. What anomaly is this?**

A) Phantom Read B) Non-Repeatable Read C) Dirty Read D) Lost Update

***Answer: C) Dirty Read.***

**Q8. The Two-Phase Locking (2PL) protocol guarantees:**

A) Deadlock prevention B) Conflict Serializability C) Maximum throughput D) No lock required

***Answer: B) Conflict Serializability.***

**Q9. In the Write-Ahead Logging (WAL) protocol, the log must be written:**

A) After the data is written to disk B) Before the data is written to disk C) After COMMIT D) Only on system crash

***Answer: B) Before the data is written to disk.***

**Q10. The SQL clause that is executed FIRST in a query's logical processing order is:**

A) WHERE B) SELECT C) FROM/JOIN D) GROUP BY

> ***Answer: C) FROM/JOIN.***

## ■ Top 10 Interview Questions

1. **Explain the ACID properties with a real-world example.**
   • *Answer:* Explain the banking transfer analogy (Alice sending Bob $100). Walk through Atomicity (rollback on crash), Consistency (total balance remains same), Isolation (Charlie can't interfere), and Durability (saved to disk).
2. **What is the difference between a B-Tree and a B+ Tree?**
   • *Answer:* A B-Tree stores data in both internal and leaf nodes. A B+ Tree stores data pointers ONLY in leaf nodes. Additionally, B+ Tree leaf nodes are linked sequentially, making range queries (`BETWEEN`) extremely fast.
3. **What is a Deadlock and how can a DBMS handle it?**
   • *Answer:* Two transactions waiting on each other's locks forever. Handled by Deadlock Detection (DBMS finds cycles in a wait-for graph and kills one transaction) or by prevention strategies like Wait-Die or Wound-Wait.
4. **If Indexing makes searching fast, why don't we index every single column?**
   • *Answer:* Indexes consume disk space, and every INSERT/UPDATE/DELETE must update the index tree too. Too many indexes slow down write operations significantly.
5. **What is a Clustered Index vs a Non-Clustered Index?**
   • *Answer:* Clustered determines physical data order, ONE per table (usually Primary Key). Non-Clustered is a separate structure with pointers back to rows; MANY per table allowed.
6. **What are the 4 Transaction Isolation Levels and what anomaly does each prevent?**
   • *Answer:* Read Uncommitted (prevents nothing), Read Committed (prevents Dirty Reads), Repeatable Read (prevents Non-Repeatable Reads, MySQL default), Serializable (prevents all anomalies including Phantom Reads).
7. **Explain the difference between Dirty Read, Non-Repeatable Read, and Phantom Read.**
   • *Answer:* Dirty Read: Reading uncommitted data. Non-Repeatable: Same row gives different values in two reads. Phantom: Same query returns different rows (due to INSERT/DELETE by another transaction).
8. **What is Write-Ahead Logging? Why is it important?**
   • *Answer:* WAL requires writing to the transaction log BEFORE writing data to disk. It enables crash recovery: REDO committed but unflused transactions, UNDO uncommitted transactions.
9. **What is the SQL Query Execution Order?**
   • *Answer:* FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. This is why you can't use SELECT aliases in WHERE.
10. **What is Two-Phase Locking (2PL)? Does it prevent deadlocks?**
    • *Answer:* 2PL has a Growing phase (acquire locks only) and Shrinking phase (release locks only). It guarantees conflict serializability but does NOT prevent deadlocks. Strict 2PL (release X-locks only at commit) prevents cascading rollbacks.

# ■■■ PostgreSQL vs MySQL — Placement Guide

This file covers key differences between PostgreSQL and MySQL, important PostgreSQL-specific features, and placement interview questions.

## 1. What is PostgreSQL?

### ■ Easy Definition

PostgreSQL (also called Postgres) is a free, open-source, advanced relational database management system.

### ■ Interview Definition

PostgreSQL is an open-source Object-Relational Database Management System (ORDBMS) that emphasizes extensibility and SQL standards compliance. It supports advanced data types, full ACID compliance, and complex queries.

> *[!NOTE]*
>
> *PostgreSQL is used by companies like Apple, Instagram, Spotify, Reddit, and Twitch. It is increasingly popular in product-based company interviews.*

## 2. PostgreSQL vs MySQL — Complete Comparison Table

| Feature | MySQL | PostgreSQL | | |
|---|---|---|---|---|
| Type | RDBMS | ORDBMS (Object-Relational) | | |
| License | Open-source (GPL) | Open-source (PostgreSQL License — more permissive) | | |
| FULL OUTER JOIN | ■ Not supported natively | ■ Supported | | |
| INTERSECT | ■ Only from v8.0.31+ | ■ Supported | | |
| EXCEPT | ■ Only from v8.0.31+ | ■ Supported | | |
| TRUNCATE rollback | ■ Auto-commits, cannot rollback | ■ Can rollback inside a transaction | | |
| Window Functions | ■ From v8.0 only | ■ Supported (much earlier) | | |
| CTEs (WITH clause) | ■ From v8.0 only | ■ Supported (much earlier) | | |
| JSON Support | JSON type (basic) | JSONB (binary, indexable, faster queries) | | |
| Arrays | ■ Not supported | ■ Native array data type | | |
| Auto-increment | AUTO_INCREMENT | SERIAL, BIGSERIAL, or GENERATED AS IDENTITY | | |
| Case sensitivity | Case-insensitive by default | Case-sensitive by default | | |
| String Concat | CONCAT(a, b) only | Both CONCAT(a, b) and `a \ | | b` operator |
| ILIKE | ■ (LIKE is case-insensitive by default) | ■ ILIKE for case-insensitive LIKE | | |
| Regex | REGEXP or RLIKE | ~ (case-sensitive), ~* (case-insensitive) | | |

| Feature | MySQL | PostgreSQL | | |
|---|---|---|---|---|
| Stored Procedures | ■ Supported | ■ Supported (also supports functions returning sets) | | |
| Performance | Faster for simple read/write | Faster for complex queries and analytics | | |
| Partial Indexes | ■ Not supported | ■ Supported | | |
| Materialized Views | ■ Not supported | ■ Supported | | |
| Inheritance | ■ Not supported | ■ Table inheritance supported | | |
| Check Constraints | Parsed but not enforced (before 8.0.16) | ■ Strictly enforced | | |

## 3. Key PostgreSQL-Only Features

### 3.1 SERIAL / AUTO-INCREMENT

```
SQL
-- MySQL way
CREATE TABLE Students (
    ID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(50)
);

-- PostgreSQL way (SERIAL)
CREATE TABLE Students (
    ID SERIAL PRIMARY KEY,
    Name VARCHAR(50)
);

-- PostgreSQL modern way (GENERATED AS IDENTITY – SQL standard)
CREATE TABLE Students (
    ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Name VARCHAR(50)
);
```

### 3.2 TRUNCATE Can Be Rolled Back

```
SQL
BEGIN;
    TRUNCATE TABLE Employees;
    -- Realize it was a mistake!
ROLLBACK;
-- In PostgreSQL: Employees table is RESTORED
-- In MySQL: Data is GONE (auto-committed)
```

### 3.3 RETURNING Clause (Very Useful!)

Get the inserted/updated/deleted rows back immediately.

```sql
-- Insert and immediately see the generated ID
INSERT INTO Students (Name) VALUES ('Alice')
RETURNING ID, Name;
-- Output: ID=1, Name=Alice

-- Delete and see what was deleted
DELETE FROM Students WHERE ID = 1
RETURNING *;
```

> *MySQL does NOT have `RETURNING`. You must use `LAST_INSERT_ID()` for inserts.*

### 3.4 JSONB — Binary JSON (PostgreSQL Superpower)

```sql
CREATE TABLE Orders (
    ID   SERIAL PRIMARY KEY,
    Info JSONB
);

INSERT INTO Orders (Info) VALUES ('{"customer": "Alice", "items": ["book", "pen"], "total": 450}');

-- Query inside JSON
SELECT Info->>'customer' AS Customer FROM Orders;
-- Output: Alice

-- Filter by JSON field
SELECT * FROM Orders WHERE Info->>'customer' = 'Alice';
```

> *MySQL's JSON type is slower for queries. PostgreSQL's JSONB stores data in binary format and can be indexed.*

### 3.5 Array Data Type

```sql
CREATE TABLE Students (
    ID    SERIAL PRIMARY KEY,
    Name  VARCHAR(50),
    Marks INT[]  -- Array of integers
);

INSERT INTO Students (Name, Marks) VALUES ('Alice', ARRAY[85, 90, 78]);

-- Query: Find students with any mark above 88
SELECT Name FROM Students WHERE 85 = ANY(Marks);
```

### 3.6 String Concatenation with ||

```sql
-- Both work in PostgreSQL
SELECT CONCAT(Emp_Name, ' - ', Dept_ID) FROM Employees;
SELECT Emp_Name || ' - ' || Dept_ID FROM Employees;

-- MySQL only supports CONCAT()
```

### 3.7 ILIKE (Case-Insensitive LIKE)

```sql
-- PostgreSQL: Find names regardless of case
SELECT * FROM Employees WHERE Emp_Name ILIKE 'alice';
-- Output: Alice (matches even though 'alice' is lowercase)

-- MySQL: LIKE is case-insensitive by default for most collations
SELECT * FROM Employees WHERE Emp_Name LIKE 'alice';
-- Output: Alice (MySQL is case-insensitive by default)
```

### 3.8 Materialized Views

A **Materialized View** stores the result physically on disk (unlike a regular view which re-runs every time).

```sql
-- Create a materialized view (stores data physically)
CREATE MATERIALIZED VIEW high_earners AS
SELECT Emp_Name, Salary FROM Employees WHERE Salary > 70000;

-- Refresh when underlying data changes
REFRESH MATERIALIZED VIEW high_earners;

-- Query like a normal table
SELECT * FROM high_earners;
```

> ***Regular View:*** *Re-runs the query every time. Slow for complex queries.*
>
> ***Materialized View:*** *Stores results. Fast to query. Must be manually refreshed.*

### 3.9 Partial Indexes

Create an index only on a subset of rows.

```sql
-- Index only on active employees (WHERE condition)
CREATE INDEX idx_active_salary ON Employees(Salary)
WHERE Salary IS NOT NULL;
-- Smaller, faster index than indexing all rows
```

---

## 4. Important Syntax Differences — Side by Side

| Operation | MySQL | PostgreSQL | | |
|---|---|---|---|---|
| Auto-increment | AUTO_INCREMENT | SERIAL or GENERATED AS IDENTITY | | |
| Limit rows | LIMIT 10 | LIMIT 10 (same) | | |
| Top N rows | LIMIT 10 | LIMIT 10 or FETCH FIRST 10 ROWS ONLY | | |
| Modify column | ALTER TABLE t MODIFY col VARCHAR(100) | ALTER TABLE t ALTER COLUMN col TYPE VARCHAR(100) | | |
| String concat | CONCAT(a, b) | `a \ | \ | b or CONCAT(a, b)` |
| Current date | NOW() or CURDATE() | NOW() or CURRENT_DATE | | |
| If-null | IFNULL(a, b) | COALESCE(a, b) (COALESCE works in both) | | |
| Show tables | SHOW TABLES; | \dt (psql) or SELECT tablename FROM pg_tables; | | |
| Show databases | SHOW DATABASES; | \l (psql) or SELECT datname FROM pg_database; | | |
| Regex match | col REGEXP 'pattern' | col ~ 'pattern' | | |
| Case-insensitive LIKE | LIKE (default) | ILIKE | | |

## 5. What Stays the Same in Both

> *[!TIP]*
>
> *For placement MCQs, if the question does not mention a specific database, these standard SQL features work identically in both.*

- All basic SELECT, WHERE, ORDER BY, LIMIT queries
- INNER JOIN, LEFT JOIN, RIGHT JOIN syntax
- GROUP BY and HAVING
- All aggregate functions (COUNT, SUM, AVG, MIN, MAX)
- Subqueries, EXISTS, IN, ANY, ALL
- CASE expressions
- COALESCE (standard SQL, works in both)
- Window functions: ROW_NUMBER(), RANK(), DENSE_RANK(), PARTITION BY (both support, MySQL from v8.0)
- CTEs with `WITH` (both support, MySQL from v8.0)
- CREATE, ALTER, DROP, INSERT, UPDATE, DELETE syntax

## 6. ■ Placement Interview Questions

### Theory Questions

**Q1. What is the difference between MySQL and PostgreSQL?**

> *MySQL is an RDBMS focused on simplicity and speed for web applications. PostgreSQL is an ORDBMS that emphasizes SQL standards compliance, extensibility, and support for advanced data types like JSONB and Arrays. Key differences: PostgreSQL supports FULL OUTER JOIN, INTERSECT, EXCEPT, TRUNCATE rollback, JSONB, Arrays, and Materialized Views natively, while MySQL does not (or added them only in recent versions).*

**Q2. Can TRUNCATE be rolled back?**

> *In PostgreSQL: YES — if TRUNCATE is executed inside a* `BEGIN...ROLLBACK` *transaction block, it can be rolled back. In MySQL: NO — TRUNCATE is auto-committed and cannot be rolled back. For placement MCQs, the standard expected answer is "TRUNCATE cannot be rolled back" (MySQL behavior).*

**Q3. What is JSONB in PostgreSQL?**

> *JSONB is a binary JSON format in PostgreSQL that stores JSON data in a decomposed binary format. It is faster to query than the regular JSON type because it doesn't need to re-parse the JSON on every read. JSONB also supports indexing on JSON fields, making searches extremely fast.*

**Q4. What is a Materialized View? How is it different from a regular View?**

> *A regular View is a virtual table — it stores the SQL query, not the data. Every time you query it, the underlying SELECT runs again. A Materialized View stores the actual result set physically on disk. It is much faster to query but requires manual refresh (*`REFRESH MATERIALIZED VIEW`*) when underlying data changes. Materialized Views are a PostgreSQL feature; MySQL does not support them.*

**Q5. What is the difference between UNION and EXCEPT in PostgreSQL?**

> *UNION combines results of two queries and removes duplicates. EXCEPT returns rows from the first query that do NOT appear in the second query result. EXCEPT is the PostgreSQL/SQL Server equivalent of Oracle's MINUS. In MySQL, EXCEPT was added only from v8.0.31.*

---

## MCQs

**Q1. Which of the following is NOT supported natively in MySQL (before v8.0)?**

A) INNER JOIN

B) FULL OUTER JOIN ■

C) LEFT JOIN

D) CROSS JOIN

> ***Answer: B) FULL OUTER JOIN.*** *MySQL does not support FULL OUTER JOIN natively. Use* `LEFT JOIN UNION RIGHT JOIN` *as a workaround.*

**Q2. In PostgreSQL, TRUNCATE inside a transaction:**

A) Cannot be rolled back

B) Can be rolled back ■

C) Causes a deadlock

D) Is not allowed inside a transaction

> ***Answer: B) Can be rolled back.*** *Unlike MySQL, PostgreSQL treats TRUNCATE as a transactional DDL statement.*

**Q3. Which PostgreSQL data type stores JSON in binary format with indexing support?**

A) JSON

B) TEXT

C) JSONB ■

D) BLOB

> ***Answer: C) JSONB.***

**Q4. What is the PostgreSQL equivalent of MySQL's `AUTO_INCREMENT`?**

A) IDENTITY

B) SEQUENCE

C) SERIAL ■

D) INCREMENT

> ***Answer: C) SERIAL*** *(or `GENERATED AS IDENTITY` in modern PostgreSQL).*

**Q5. Which clause is unique to PostgreSQL that returns the affected rows after INSERT/UPDATE/DELETE?**

A) OUTPUT

B) RETURNING ■

C) FETCH

D) RESULT

> ***Answer: B) RETURNING.*** *SQL Server uses `OUTPUT`. MySQL has no equivalent.*

**Q6. Which of the following is a case-insensitive LIKE operator in PostgreSQL?**

A) LIKE

B) RLIKE

C) ILIKE ■

D) SLIKE

> ***Answer: C) ILIKE.*** *MySQL's LIKE is case-insensitive by default, so it doesn't need ILIKE.*

**Q7. What does `REFRESH MATERIALIZED VIEW` do?**

A) Creates a new materialized view

B) Deletes a materialized view

C) Updates the stored data of a materialized view with fresh query results ■

D) Converts a view to a materialized view

> ***Answer: C.***

**Q8. Which PostgreSQL operator is used for pattern matching with regex (case-sensitive)?**

A) REGEXP

B) RLIKE

C) ~ ■

D) MATCH

> ***Answer: C) ~** (tilde). Use `~*` for case-insensitive. MySQL uses `REGEXP`.*

---

## 7. ■ 2-Minute Quick Revision

1. **PostgreSQL = ORDBMS** (Object-Relational). MySQL = RDBMS.
2. **FULL OUTER JOIN** — PostgreSQL ■ | MySQL ■ (use LEFT UNION RIGHT).
3. **INTERSECT / EXCEPT** — PostgreSQL ■ | MySQL from v8.0.31+ only.
4. **TRUNCATE rollback** — PostgreSQL ■ | MySQL ■.
5. **JSONB** — PostgreSQL's binary JSON. Faster, indexable. MySQL has basic JSON only.
6. **RETURNING** — PostgreSQL's way to get back rows after INSERT/UPDATE/DELETE.
7. **SERIAL** — PostgreSQL's auto-increment. MySQL uses `AUTO_INCREMENT`.
8. **ILIKE** — PostgreSQL's case-insensitive LIKE.
9. **||** — PostgreSQL string concat operator. MySQL only has `CONCAT()`.
10. **Materialized Views** — PostgreSQL stores results physically. Must `REFRESH`. MySQL doesn't have this.
11. **Arrays** — PostgreSQL supports array columns natively. MySQL doesn't.
12. **Partial Indexes** — PostgreSQL can index a subset of rows. MySQL can't.
13. **Standard SQL (joins, aggregates, window functions, CTEs, CASE)** — Same in both.

# ■ Ultimate Revision and Practice Handbook ■■■■■

This resource consolidates all the high-level study sheets, master practice question sets, and last-minute revision notes for placement preparation.

## 1. The Ultimate DBMS Revision Sheet ■■■■■

### ■■ Schema & Structural Design

- **Three Abstraction Levels:** View (External) -> Conceptual (Logical) -> Internal (Physical).
- **Data Independence:**
  - *Physical:* Modify physical storage (SSD vs. HDD, index files) without affecting logical schemas.
  - *Logical:* Modify conceptual schemas (adding attributes, partitioning tables) without affecting user views.
- **Metadata Catalog:** A set of read-only system tables containing database structure definitions.

### ■ Keys Cheat Sheet

- **Super Key (SK):** Any set of columns that uniquely identifies a row.
- **Candidate Key (CK):** A minimal super key. Removing any column destroys uniqueness.
- **Primary Key (PK):** The candidate key selected to uniquely identify rows. Cannot contain NULLs.
- **Alternate Key (AK):** All candidate keys not selected as the primary key ($AK = CK - PK$).
- **Foreign Key (FK):** Points to a primary key in a parent table. Enforces referential integrity.
- **Composite Key:** A key composed of more than one column.

### ■ Normalization Cheat Sheet

- **1NF:** Atomic attribute values only. No composite or multivalued attributes.
- **2NF:** In 1NF + No **Partial Dependency** ($\text{Part of Candidate Key} \rightarrow \text{Non-Prime}$).
- **3NF:** In 2NF + No **Transitive Dependency** ($X \rightarrow Y \Rightarrow X \text{ is Super Key or } Y \text{ is Prime}$).
- **BCNF:** For every non-trivial $X \rightarrow Y$, $X$ must be a Super Key.
- **Lossless-Join Test:** $R_1 \cap R_2 \rightarrow R_1 \text{ or } R_1 \cap R_2 \rightarrow R_2$.
- **Dependency Preservation:** $(F_1 \cup F_2)^+ \equiv F^+$.

### ■■ Concurrency & Locks

- **ACID:** Atomicity (Transaction Manager), Consistency (Programmer/Constraints), Isolation (Concurrency Control), Durability (Recovery Manager).
- **2PL:** Growing phase (locks acquired, none released) -> Shrinking phase (locks released, none acquired).
- **Strict 2PL:** Hold all Exclusive locks until commit to prevent cascading rollbacks.
- **Deadlock Prevention:** Wait-Die (older waits, younger dies) and Wound-Wait (older aborts younger, younger waits).

## 2. The Ultimate SQL Revision Sheet ■■■■■

### ■ Logical Query Execution Order

$$\text{FROM} \rightarrow \text{JOIN} \rightarrow \text{ON} \rightarrow \text{WHERE} \rightarrow \text{GROUP BY} \rightarrow \text{HAVING} \rightarrow \text{SELECT} \rightarrow \text{DISTINCT} \rightarrow \text{ORDER BY} \rightarrow \text{LIMIT}$$

### ■ Core Syntax Templates

**Joins**

- **Inner Join:** Matches keys in both tables.
- **Left Join:** All rows from left table, with NULLs for non-matching right columns.
- **Self Join:** Join a table to itself using distinct aliases (e.g., matching employees to managers).

**Window Functions**

- `ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC)`
- `RANK() OVER (ORDER BY salary DESC)` (leaves gaps after ties: `1, 2, 2, 4`)
- `DENSE_RANK() OVER (ORDER BY salary DESC)` (no gaps: `1, 2, 2, 3`)
- `LAG(salary, 1)` (retrieves value from the previous row)
- `LEAD(salary, 1)` (retrieves value from the next row)

**Null Handling**

- `COALESCE(val1, val2, 0)`: Returns the first non-null value.
- `IS NULL` / `IS NOT NULL`: Used to check for NULL values in filters.

---

## ■ Top 100 Placement MCQs ■■■■■

*This set contains a mixed-bag of questions covering DBMS, ER models, normalization, SQL, and advanced concurrency.*

**Q1. Which level of database schema abstraction describes how data is physically stored on disk?**

A) Conceptual Level

B) External Level

C) Internal Level ■

D) Logical Level

*Explanation: The internal (physical) level defines paths, indexing, and block sizes on storage.*

**Q2. A candidate key that is not chosen as the primary key is called:**

A) Super Key

B) Foreign Key

C) Alternate Key ■

D) Composite Key

*Explanation: Alternate keys are candidate keys not selected as the primary key.*

**Q3. What constraint requires foreign key values in a child table to match primary key values in the parent table?**

A) Entity Integrity

B) Referential Integrity ■

C) Domain Constraint

D) Check Constraint

*Explanation: Referential integrity enforces valid relationships between foreign and primary keys.*

**Q4. What occurs when deleting a row deletes unrelated but important structural data?**

A) Deletion Anomaly ■

B) Update Anomaly

C) Insertion Anomaly

D) Lost Update

*Explanation: Deletion anomaly is the accidental loss of unrelated facts when a row is deleted.*

**Q5. A relation is in 2NF if it is in 1NF and contains no:**

A) Transitive dependencies

B) Partial dependencies ■

C) Multi-valued dependencies

D) Join dependencies

*Explanation: 2NF eliminates partial dependencies (non-prime attributes depending on a subset of a candidate key).*

**Q6. For a non-trivial FD X -> Y, if X is not a super key but Y is a prime attribute, the relation is in:**

A) BCNF

B) 2NF but not 3NF

C) 3NF ■

D) 1NF only

*Explanation: 3NF allows X -> Y if Y is prime, whereas BCNF requires X to be a super key.*

**Q7. Which property ensures that a transaction is rolled back if any statement fails?**

A) Isolation

B) Consistency

C) Atomicity ■

D) Durability

*Explanation: Atomicity guarantees "all-or-nothing" transaction execution.*

**Q8. What log protocol requires writing modifications to log files on disk before updating data tables?**

A) Write-Ahead Logging (WAL) ■

B) Checkpoint Logging

C) Commit Logging

D) Rollback Buffering

*Explanation: WAL requires log entries to be written to disk before database modifications are applied.*

**Q9. Which isolation level prevents dirty reads but allows non-repeatable reads?**

A) Read Uncommitted

B) Read Committed ■

C) Repeatable Read

D) Serializable

*Explanation: Read Committed restricts reads to committed data, preventing dirty reads.*

**Q10. Which window function assigns sequential numbers starting at 1?**

A) RANK()

B) DENSE_RANK()

C) ROW_NUMBER() ■

D) NTILE()

*Explanation: ROW_NUMBER() assigns consecutive integers to rows.*

**Q11. Which SQL operator matches values against a wildcard pattern?**

A) IN

B) LIKE ■

C) BETWEEN

D) EXISTS

*Explanation: LIKE performs pattern matching using % and _.*

**Q12. In an ER diagram, a multivalued attribute is represented by:**

A) Oval

B) Dashed Oval

C) Double Oval ■

D) Rectangle

*Explanation: Double ovals represent multivalued attributes.*

**Q13. An entity set that does not have sufficient attributes to form a primary key is a:**

A) Strong Entity Set

B) Weak Entity Set ■

C) Simple Entity Set

D) Composite Entity Set

*Explanation: Weak entity sets lack a primary key and depend on a strong parent entity.*

**Q14. What occurs when a transaction reads uncommitted changes that are later rolled back?**

A) Non-repeatable read

B) Dirty read ■

C) Phantom read

D) Lost update

*Explanation: Dirty reads occur when uncommitted data is read by another transaction.*

**Q15. How many clustered indexes can a table have?**

A) 1 ■

B) 2

C) 32

D) Unlimited

*Explanation: Because physical tables can be sorted in only one way, there can be at most one clustered index.*

**Q16. Which data model organizes records in a tree structure where each child has only one parent?**

A) Network Model

B) Relational Model

C) Hierarchical Model ■

D) Object-Oriented Model

*Explanation: Hierarchical models use tree structures; child nodes can have only one parent.*

**Q17. The database schema represents:**

A) The physical files on disk

B) The state of database data at a given point in time

C) The logical design or blueprint of the database ■

D) The user privilege log

*Explanation: The schema is the structural design of the database.*

**Q18. Which command is used to add a new column to an existing table?**

A) UPDATE

B) ALTER TABLE ■

C) CREATE COLUMN

D) ADD COLUMN

*Explanation: Modifying structural definitions is a DDL operation performed via `ALTER TABLE`.*

**Q19. What does `SELECT COUNT(dept_id) FROM employees` return?**

A) The total number of rows in the table

B) The number of rows with non-null department IDs ■

C) The count of unique departments

D) The sum of department IDs

*Explanation: Passing a column name to COUNT ignores NULL values.*

**Q20. What is a minimal super key?**

A) Primary Key

B) Candidate Key ■

C) Alternate Key

D) Foreign Key

*Explanation: Candidate keys are minimal super keys.*

**Q21. Which join type returns all rows from the right table, plus matching rows from the left?**

A) LEFT OUTER JOIN

B) RIGHT OUTER JOIN ■

C) INNER JOIN

D) FULL OUTER JOIN

*Explanation: RIGHT JOIN returns all rows from the right table, with NULLs for non-matching left columns.*

**Q22. The intersection of R1 and R2 must determine either R1 or R2 for a decomposition to be:**

A) Dependency-preserving

B) Lossless-join ■

C) BCNF

D) Trivial

*Explanation: The common attributes (intersection) must form a key for at least one of the decomposed tables.*

**Q23. Which command is a DML command?**

A) CREATE

B) UPDATE ■

C) ALTER

D) TRUNCATE

*Explanation: UPDATE modifies data values, making it a DML command.*

**Q24. What shape represents a relationship in an ER diagram?**

A) Rectangle

B) Oval

C) Diamond ■

D) Double Oval

*Explanation: Diamonds represent relationships in Peter Chen's ER notation.*

**Q25. Which anomaly occurs when a transaction overwrites another transaction's updates without reading them?**

A) Lost Update ■

B) Dirty Read

C) Non-Repeatable Read

D) Phantom Read

*Explanation: Lost update occurs when concurrent writes overwrite changes without isolating them.*

**Q26. Which isolation level is the strongest and prevents all anomalies?**

A) Repeatable Read

B) Serializable ■

C) Read Committed

D) Snapshot Isolation

*Explanation: Serializable is the highest isolation level, executing transactions as if they were run sequentially.*

**Q27. A functional dependency X -> Y is trivial if:**

A) Y is NULL

B) Y is a subset of X ■

C) X is a primary key

D) Y determines X

*Explanation: Trivial FDs are automatically true by definition because the RHS is a subset of the LHS.*

**Q28. What shape represents a weak entity set in an ER diagram?**

A) Rectangle

B) Double Rectangle ■

C) Diamond

D) Oval

*Explanation: Double rectangles represent weak entities.*

**Q29. Which command is used to remove all rows from a table and deallocate its pages?**

A) DELETE

B) TRUNCATE ■

C) DROP

D) REMOVE

*Explanation: TRUNCATE is a DDL command that resets a table, bypassing delete triggers.*

**Q30. What does the COALESCE function return?**

A) The number of non-null records

B) The first non-null value in a list of arguments ■

C) The string length

D) The sum of columns

*Explanation: `COALESCE(val1, val2, ...)` returns the first non-NULL expression.*

**Q31. In the database query execution order, which clause is evaluated first?**

A) SELECT

B) WHERE

C) FROM ■

D) GROUP BY

*Explanation: FROM is evaluated first to identify source tables.*

**Q32. What occurs during the growing phase of 2PL?**

A) The database is compressed

B) Transactions can acquire locks but cannot release any ■

C) Transactions abort

D) Index files shrink

*Explanation: In the growing phase, locks can be acquired but none can be released.*

**Q33. B+ Tree leaf nodes are connected via a:**

A) Linked List ■

B) Binary Tree

C) Stack

D) Queue

*Explanation: Leaf nodes are linked as a doubly linked list to optimize sequential scans.*

**Q34. OLAP databases are typically:**

A) Normalized to 3NF

B) Denormalized ■

C) Stored in RAM only

D) Free of indexes

*Explanation: OLAP systems use denormalized schemas (like Star or Snowflake) to speed up complex queries.*

**Q35. Distributing database partitions across multiple physical servers is called:**

A) Partitioning

B) Sharding ■

C) Replication

D) Normalization

*Explanation: Sharding distributes partitions across multiple database servers.*

**Q36. MongoDB is which type of NoSQL database?**

A) Key-Value

B) Document-oriented ■

C) Column Family

D) Graph Database

*Explanation: MongoDB stores records as documents (BSON).*

**Q37. What is a self-join?**

A) A join between two instances of the same table ■

B) A join that matches primary keys with itself

C) A join that runs without ON conditions

D) A join that updates records

*Explanation: A self-join joins a table to itself using distinct aliases.*

**Q38. How is TRUNCATE different from DELETE?**

A) TRUNCATE can have a WHERE clause

B) DELETE is faster than TRUNCATE

C) TRUNCATE is a DDL command that deallocates storage pages, bypassing delete triggers ■

D) TRUNCATE can be rolled back in all engines

*Explanation: TRUNCATE is a DDL operation that resets a table, making it faster than row-by-row DELETE operations.*

**Q39. What does the EXISTS operator return?**

A) A table of matching values

B) A boolean value indicating if a subquery returns any rows ■

C) A list of non-null IDs

D) The count of records

*Explanation: EXISTS returns TRUE if the subquery returns at least one row, stopping evaluation early.*

**Q40. Which window function assigns ranks with gaps?**

A) RANK() ■

B) DENSE_RANK()

C) ROW_NUMBER()

D) LEAD()

*Explanation: RANK() leaves gaps in ranks in case of ties (e.g., `1, 2, 2, 4`).*

**Q41. In the relational model, relations are represented as:**

A) Trees

B) Graphs

C) Tables ■

D) Files

*Explanation: Relational database design represents relations as two-dimensional tables.*

**Q42. What is metadata?**

A) Big data

B) Unstructured data

C) Data about data ■

D) Encrypted data

*Explanation: Metadata is structural information that describes schemas, tables, columns, and constraints.*

**Q43. What rule prevents primary keys from containing NULL values?**

A) Referential Integrity

B) Entity Integrity ■

C) Domain Constraint

D) Key Constraint

*Explanation: Entity Integrity states that no primary key attribute can be NULL to ensure unique row identification.*

**Q44. The ability to modify the conceptual schema without rewriting application programs is called:**

A) Physical data independence

B) Logical data independence ■

C) Schema evolution

D) Structural integrity

*Explanation: Logical data independence protects the external view (applications) from conceptual changes.*

**Q45. Which level of DBMS architecture is closest to the physical storage?**

A) External level

B) View level

C) Conceptual level

D) Internal level ■

*Explanation: The internal level (physical schema) describes how data blocks, files, and indices are stored on disk.*

**Q46. In Three-Level Architecture, mappings are used to:**

A) Link different user applications

B) Translate requests between levels to achieve data independence ■

C) Compress files on physical disks

D) Log query statistics

*Explanation: Mappings connect the external-conceptual and conceptual-internal levels to maintain data independence.*

**Q47. A composite key is a key that consists of:**

A) Text and numeric columns

B) References to multiple parent tables

C) Multiple attributes ■

D) Encrypted values

*Explanation: Composite keys combine two or more columns to form a unique identifier.*

**Q48. Which database user is responsible for authorization management and backup configuration?**

A) Database Designer

B) Application Developer

C) Database Administrator (DBA) ■

D) Naive User

*Explanation: DBAs manage access permissions, backups, performance tuning, and hardware management.*

**Q49. What is the main disadvantage of the Hierarchical Data Model?**

A) It lacks graphical tools

B) It cannot support 1:N relations

C) It cannot support N:M (many-to-many) relationships naturally ■

D) It consumes too much CPU

*Explanation: Trees only allow a child node to have one parent, making N:M relations hard to model.*

**Q50. A database state that violates referential integrity contains:**

A) Duplicate primary key values

B) A foreign key value that does not exist as a primary key value in the parent table ■

C) Two rows with identical values

D) A candidate key that is NULL

*Explanation: Referential integrity requires foreign keys to match an existing primary key or be NULL.*

**Q51. What occurs during ON DELETE CASCADE?**

A) Deleting a child row deletes the parent row

B) Deleting a parent row automatically deletes its associated child rows ■

C) Deleting a parent row sets child foreign keys to NULL

D) The delete operation is blocked

*Explanation: CASCADE propagates parent-row deletions downwards to child tables.*

**Q52. Which shape represents a derived attribute in an ERD?**

A) Oval

B) Double Oval

C) Dashed Oval ■

D) Rectangle

*Explanation: Dashed ovals represent derived attributes.*

**Q53. If A -> B and B -> C, then A -> C. Which axiom is this?**

A) Augmentation

B) Reflexivity

C) Transitivity ■

D) Decomposition

*Explanation: Transitivity rule: if A determines B and B determines C, then A determines C.*

**Q54. In 3NF, for every non-trivial FD X -> Y, which condition must hold?**

A) X is a super key or Y is a prime attribute ■

B) Y must be a super key

C) X must be a prime attribute

D) X and Y must be composite

*Explanation: 3NF allows X -> Y if X is a super key OR if Y is a prime attribute.*

**Q55. BCNF is stricter than 3NF because:**

A) It does not allow primary keys

B) For every non-trivial FD X -> Y, X must be a super key ■

C) It allows multivalued dependencies

D) It requires foreign key indexes

*Explanation: BCNF removes the "Y is a prime attribute" exception, requiring the LHS of all non-trivial FDs to be a super key.*

**Q56. Prime attributes are attributes that:**

A) Are numeric prime numbers

B) Are part of some candidate key ■

C) Are foreign keys

D) Can be NULL

*Explanation: An attribute is prime if it is a member of any candidate key.*

**Q57. If a table contains employee skills, and an employee can have multiple skills, storing them in a single row as "C++, Java" violates:**

A) 1NF ■

B) 2NF

C) 3NF

D) BCNF

*Explanation: Comma-separated values are non-atomic, which violates 1NF.*

**Q58. A table with candidate key A and FD B -> C has a:**

A) Partial dependency

B) Transitive dependency ■

C) No dependency

D) Primary dependency

*Explanation: A determines B (since A is a key) and B determines C. This yields a transitive dependency: A -> B -> C.*

**Q59. What shape represents a relationship in an ERD?**

A) Oval

B) Rectangle

C) Diamond ■

D) Double Oval

*Explanation: Diamonds represent relationships.*

**Q60. Which normal form is concerned with multi-valued dependencies (MVD)?**

A) 3NF

B) BCNF

C) 4NF ■

D) 5NF

*Explanation: 4NF is designed to eliminate multi-valued dependencies.*

**Q61. Which clause is used to filter records after aggregation?**

A) WHERE

B) HAVING ■

C) GROUP BY

D) ORDER BY

*Explanation: HAVING filters groups; WHERE filters records before grouping.*

**Q62. What does the UNION operator do?**

A) Combines tables side-by-side

B) Combines the result sets of two queries and removes duplicates ■

C) Combines result sets keeping all duplicates

D) Sorts two tables

*Explanation: UNION combines query outputs vertically and removes duplicate rows.*

**Q63. What is the difference between UNION and UNION ALL?**

A) UNION is faster than UNION ALL

B) UNION removes duplicates, while UNION ALL keeps all records ■

C) UNION works on strings, UNION ALL on numbers

D) UNION ALL can only combine three queries

*Explanation: UNION performs a deduplication step, making UNION ALL faster.*

**Q64. Which join returns all records from the left table and matching records from the right?**

A) INNER JOIN

B) RIGHT JOIN

C) LEFT JOIN ■

D) CROSS JOIN

*Explanation: LEFT JOIN includes all rows from the left table, with NULLs for non-matching right table columns.*

**Q65. What is a Cartesian Product in SQL?**

A) A join that matches keys

B) The result of a LEFT JOIN

C) The pairing of every row in one table with every row in another (CROSS JOIN) ■

D) A query that returns zero rows

*Explanation: A CROSS JOIN produces a Cartesian product, returning $M \times N$ rows.*

**Q66. How do you check for NULL values in a WHERE clause?**

A) `column = NULL`

B) `column IS NULL` ■

C) `column IN (NULL)`

D) `column EQUALS NULL`

*Explanation: Direct comparison operators with NULL fail. Use `IS NULL` instead.*

**Q67. What is a CTE in SQL?**

A) Constant Table Entity

B) Common Table Expression ■

C) Column Transaction Engine

D) Core Table Extension

*Explanation: A CTE is a temporary named result set defined using the `WITH` clause.*

**Q68. Which window function assigns sequential ranks without gaps?**

A) RANK()

B) DENSE_RANK() ■

C) ROW_NUMBER()

D) LEAD()

*Explanation: DENSE_RANK() assigns consecutive integer ranks. RANK() leaves gaps.*

**Q69. Which constraint ensures that a column cannot have duplicate values?**

A) NOT NULL

B) CHECK

C) UNIQUE ■

D) DEFAULT

*Explanation: The UNIQUE constraint guarantees that all values in a column are distinct.*

**Q70. What is a correlated subquery?**

A) A subquery that runs once before the outer query

B) A subquery that references columns from the outer query, executing once for each row in the outer query ■

C) A subquery that uses joins

D) A subquery defined inside a CTE

*Explanation: Correlated subqueries depend on the outer query's current row value.*

**Q71. Can a PRIMARY KEY column contain NULL values?**

A) Yes, if it is a composite key

B) No ■

C) Yes, at most one row

D) Yes, if configured in the schema

*Explanation: Primary keys enforce unique, non-null properties.*

**Q72. What is the output of `SELECT 5 + NULL`?**

A) 5

B) 0

C) NULL ■

D) Error

*Explanation: Any arithmetic operation involving a NULL value yields NULL.*

**Q73. What is the purpose of the FOREIGN KEY constraint?**

A) Enforces entity uniqueness

B) Speed up query execution

C) Enforces referential integrity between tables ■

D) Compresses data

*Explanation: Foreign keys link tables, ensuring that child table values point to valid parent records.*

**Q74. Which command is used to remove a table structure and all its data?**

A) DELETE TABLE

B) TRUNCATE TABLE

C) DROP TABLE ■

D) REMOVE TABLE

*Explanation: DROP deletes the table structure, data, indexes, and constraints.*

**Q75. What does the window partition clause `PARTITION BY` do?**

A) Physically partitions the table on disk

B) Groups rows into partitions to calculate window function values independently within each group ■

C) Sorts the final output

D) Deletes duplicate groups

*Explanation: `PARTITION BY` divides rows into groups for window function computation.*

**Q76. Which function finds the difference between consecutive records?**

A) FIRST_VALUE()

B) LEAD() / LAG() ■

C) ROW_NUMBER()

D) NTILE()

*Explanation: LEAD() and LAG() access values in rows relative to the current row.*

**Q77. What is a VIEW in SQL?**

A) A physical table copy stored on disk

B) A virtual table defined by a saved SQL query ■

C) A performance-tuning index

D) A database trigger

*Explanation: Views are virtual tables that run their defined SELECT query when queried.*

**Q78. Which set operator returns records from the first query that are not present in the second?**

A) UNION

B) INTERSECT

C) EXCEPT / MINUS ■

D) UNION ALL

*Explanation: EXCEPT (or MINUS) returns the set difference.*

**Q79. Can a table have more than one UNIQUE constraint?**

A) Yes ■

B) No

C) Only if there is no primary key

D) Only on numeric columns

*Explanation: Yes, a table can have multiple unique columns, but at most one primary key.*

**Q80. What does the wildcard _ represent?**

A) Zero or more characters

B) Exactly one character ■

C) A number

D) A space

*Explanation: The underscore matches exactly one character in LIKE patterns.*

**Q81. What does the wildcard % represent?**

A) Exactly one character

B) Zero or more characters ■

C) A percentage number

D) A null character

*Explanation: The percent symbol matches any string of zero or more characters.*

**Q82. Which JOIN returns all rows from the right table?**

A) LEFT JOIN

B) RIGHT JOIN ■

C) INNER JOIN

D) FULL JOIN

*Explanation: RIGHT JOIN returns all rows from the right table.*

**Q83. Which constraint prevents null values in a column?**

A) UNIQUE

B) NOT NULL ■

C) CHECK

D) DEFAULT

*Explanation: `NOT NULL` prevents NULL values.*

**Q84. Which function returns the row count?**

A) SUM()

B) COUNT() ■

C) ROW_NUMBER()

D) SIZE()

*Explanation: `COUNT()` counts rows.*

**Q85. Which CAP guarantee ensures that every read returns the most recent write?**

A) Consistency ■

B) Availability

C) Partition Tolerance

D) Speed

*Explanation: Consistency guarantees that reads return the latest data or fail.*

**Q86. Key-Value stores are best suited for:**

A) Complex reports

B) Simple caching and session management ■

C) Social network graphs

D) Document indexing

*Explanation: Key-Value stores are optimized for simple lookups, making them ideal for caching.*

**Q87. A transaction that is rolled back enters the:**

A) Failed State

B) Aborted State ■

C) Terminated State

D) Suspended State

*Explanation: Aborted is the state entered after rollback is complete.*

**Q88. Which anomaly does Read Committed isolation allow?**

A) Dirty Read

B) Non-Repeatable Read ■

C) Lost Update

D) Transaction deadlock

*Explanation: Read Committed allows non-repeatable reads and phantom reads.*

**Q89. What does a Wait-For Graph (WFG) show?**

A) CPU wait times

B) Transaction lock dependencies ■

C) Log buffer write delays

D) Query queues

*Explanation: WFGs map transactions and the locks they are waiting for.*

**Q90. What occurs during vertical partitioning?**

A) Rows are split

B) Columns are split into separate tables ■

C) Tables are distributed across servers

D) Indexes are deleted

*Explanation: Vertical partitioning splits columns into separate tables.*

**Q91. In MongoDB, a row equivalent is called a:**

A) Collection

B) Document ■

C) Field

D) Object

*Explanation: Documents represent individual records in MongoDB.*

**Q92. Strict 2-Phase Locking prevents:**

A) Deadlocks

B) Cascading Rollbacks ■

C) Thread leaks

D) Query syntax errors

*Explanation: Strict 2PL prevents cascading rollbacks by holding exclusive locks until transactions commit.*

**Q93. The Wait-Die scheme is based on:**

A) Lock types

B) Transaction ages (timestamps) ■

C) Query complexity

D) Table row counts

*Explanation: Wait-Die is a deadlock prevention protocol based on transaction age.*

**Q94. In timestamp-based concurrency control, transactions are ordered by:**

A) Priority numbers

B) Start times (timestamps) ■

C) Execution costs

D) The DBA

*Explanation: Timestamp protocols order transactions by their start times.*

**Q95. Which database system is optimized for high volumes of simple transaction writes and reads?**

A) OLAP

B) NoSQL Graph DB

C) OLTP ■

D) Data Warehouse

*Explanation: Online Transaction Processing (OLTP) is optimized for real-time transaction processing.*

**Q96. How does a DBMS detect deadlocks?**

A) By parsing queries

B) By using a Wait-For Graph (WFG) ■

C) By measuring CPU load

D) By tracking transaction row counts

*Explanation: The DBMS checks for cycles in the WFG to detect deadlocks.*

**Q97. Which normal form is concerned with join dependencies?**

A) 3NF

B) BCNF

C) 4NF

D) 5NF ■

*Explanation: 5NF (Project-Join Normal Form) deals with join dependencies.*

**Q98. What does `SELECT COUNT(*)` do?**

A) Counts columns

B) Counts all rows, including those with NULL values ■

C) Counts unique values

D) Sums numeric fields

*Explanation: `COUNT(*)` counts the total number of rows.*

**Q99. What does the UNION operator do to duplicate rows?**

A) Keeps them

B) Removes them ■

C) Sorts them

D) Throws an error

*Explanation: UNION removes duplicates.*

**Q100. Which clause is evaluated last in a SELECT query?**

A) SELECT

B) ORDER BY

C) LIMIT ■

D) HAVING

*Explanation: LIMIT is evaluated last to restrict the returned row count.*

---

## ■ Top 100 Interview Questions ■■■■■

**Q1. Explain the difference between DBMS and File System.**

**Answer:** File systems store raw files on disk without transactional safety, query optimization, or fine-grained concurrency control. A DBMS provides structured tables, ACID compliance, lock-based concurrency control, indexing, and declarative query languages (SQL).

**Q2. What is Data Abstraction? Explain the three levels of database abstraction.**

**Answer:** Data abstraction hides physical storage details. The three levels are:

1. *Internal (Physical):* Describes physical storage structures (files, block layouts, B+ Trees).
2. *Conceptual (Logical):* Describes tables, columns, relations, and constraints.
3. *External (View):* Describes subsets of data visible to specific users.

**Q3. What is the difference between Physical and Logical Data Independence?**

**Answer:**

- *Physical:* Modify physical storage (SSD vs. HDD, index files) without affecting logical schemas.
- *Logical:* Modify conceptual schemas (adding columns, splitting tables) without affecting user views or application code.

**Q4. What is a Data Dictionary/System Catalog?**

**Answer:** A read-only set of system tables where the DBMS stores metadata (schema definitions, table properties, constraints, indexing directories, and user privileges).

**Q5. Define Super Key, Candidate Key, and Primary Key. Give an example.**

**Answer:**

- *Super Key:* Any set of columns that uniquely identifies a row.
- *Candidate Key:* A minimal super key (no redundant columns).
- *Primary Key:* The selected candidate key used to identify rows. Cannot contain NULLs.
- *Example:* In `Employee(EmpID, PassportNo, Name)`, `EmpID` and `PassportNo` are candidate keys. If `EmpID` is chosen as the primary key, `PassportNo` is the alternate key.

**Q6. What is a Foreign Key? Why is it used?**

**Answer:** A column in a table that references the primary key of another table. It is used to link tables and enforce referential integrity.

**Q7. Explain Entity Integrity and Referential Integrity constraints.**

**Answer:**

- *Entity Integrity:* Primary key columns cannot contain NULL values.
- *Referential Integrity:* Foreign key values must either match a valid primary key value in the parent table or be NULL.

**Q8. What are the different referential actions available during delete/update operations?**

**Answer:** `ON DELETE/UPDATE CASCADE` (propagate changes), `SET NULL` (set foreign keys to NULL), `SET DEFAULT` (set foreign keys to default values), and `RESTRICT` / `NO ACTION` (block the operation).

**Q9. Difference between DDL, DML, and TCL.**

**Answer:**

- *DDL (Data Definition):* Defines structure (`CREATE`, `ALTER`, `DROP`, `TRUNCATE`). Auto-committed.
- *DML (Data Manipulation):* Writes data (`INSERT`, `UPDATE`, `DELETE`). Can be rolled back.
- *TCL (Transaction Control):* Manages transactions (`COMMIT`, `ROLLBACK`, `SAVEPOINT`).

**Q10. What is a composite key? When would you use it?**

**Answer:** A key consisting of two or more columns to guarantee uniqueness. Used in junction tables for many-to-many relationships (e.g., `StudentCourses(StudentID, CourseID)`).

**Q11. Compare DELETE, TRUNCATE, and DROP commands.**

**Answer:**

- *DELETE:* DML. Deletes specific rows using a WHERE clause. Writes to logs (slow) and can be rolled back.
- *TRUNCATE:* DDL. Empties the table by deallocating pages. Fast, auto-committed, and bypasses delete triggers.
- *DROP:* DDL. Permanently deletes the table structure and data.

**Q12. What are the advantages of the Relational Model over Hierarchical and Network models?**

**Answer:** Relational models represent data in simple 2D tables, which is easier to understand than trees or graph pointers. Relationships are defined by data values (Keys), not memory pointers, providing physical data independence.

**Q13. Explain the difference between a Strong Entity Set and a Weak Entity Set.**

**Answer:** Strong entities have their own primary key. Weak entities lack a primary key and depend on a strong parent entity (owner) for identification using a discriminator (partial key).

**Q14. What are the different types of attributes in an ER model?**

**Answer:** Simple (atomic), Composite (divisible), Multivalued (multiple values, double oval), and Derived (computed, dashed oval).

**Q15. What are Cardinality Ratios and Participation Constraints in ER Diagrams?**

**Answer:** Cardinality defines maximum relationship links (1:1, 1:N, M:N). Participation defines minimum relationship links: Total (every entity participates, double line) or Partial (only some participate, single line).

**Q16. What is Functional Dependency?**

**Answer:** A constraint $X \rightarrow Y$ stating that if two rows match on value $X$, they must match on value $Y$ (X functionally determines Y).

### Q17. Explain Armstrong's Axioms.

**Answer:** Primary rules for deriving FDs: Reflexivity (if $Y \subseteq X$, then $X \rightarrow Y$), Augmentation (if $X \rightarrow Y$, then $XZ \rightarrow YZ$), and Transitivity (if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$).

### Q18. What is the closure of an attribute set?

**Answer:** The set of all attributes functionally determined by that set under a given set of functional dependencies $F$ (denoted $X^+$).

### Q19. How do you find the Candidate Keys of a relation using attribute closure?

**Answer:** Identify attributes that never appear on the RHS of any FD. Find their closure. If it contains all attributes of the relation, they form the candidate key. Otherwise, systematically add other columns to find minimal combinations that determine all attributes.

### Q20. What is a Minimal Cover (or Canonical Cover)?

**Answer:** A simplified, equivalent set of dependencies containing no redundant FDs or redundant attributes (decomposed RHS, removed extraneous LHS attributes, and removed redundant FDs).

### Q21. Define Database Normalization. Why is it performed?

**Answer:** The process of organizing database tables to minimize redundancy and eliminate insertion, update, and deletion anomalies.

### Q22. Explain the difference between 3NF and BCNF.

**Answer:** For a non-trivial FD $X \rightarrow Y$, 3NF allows $X$ to be a super key OR $Y$ to be a prime attribute. BCNF requires $X$ to be a super key (no prime attribute exception).

### Q23. Can we always decompose a relation into BCNF? What is the trade-off?

**Answer:** We can always decompose any relation into BCNF in a lossless manner. However, BCNF decomposition is not always dependency-preserving.

### Q24. What is Lossless-Join Decomposition? How do you test for it?

**Answer:** A decomposition of relation $R$ into $R_1$ and $R_2$ is lossless-join if joining the sub-relations reconstructs the original table without creating fake rows. Test: $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$.

### Q25. What is Dependency Preservation?

**Answer:** A decomposition is dependency-preserving if the union of the functional dependencies of the sub-relations is equivalent to the original set of dependencies $F$, allowing constraints to be verified within individual tables.

### Q26. Explain SQL logical query execution order.

**Answer:** `FROM` -> `JOIN` -> `ON` -> `WHERE` -> `GROUP BY` -> `HAVING` -> `SELECT` -> `DISTINCT` -> `ORDER BY` -> `LIMIT`.

### Q27. What is the difference between WHERE and HAVING?

**Answer:** `WHERE` filters rows before grouping and cannot contain aggregate functions. `HAVING` filters groups after grouping and can contain aggregate functions.

**Q28. Compare JOIN types.**
**Answer:** `INNER` (matching keys), `LEFT` (all left, matching right), `RIGHT` (all right, matching left), `FULL` (all rows from both), and `CROSS` (Cartesian product).

**Q29. What is a Self Join? Give a use case.**
**Answer:** A table joined to itself using distinct aliases (e.g., finding managers of employees when both are stored in the same table).

**Q30. What is the difference between a Subquery and a Correlated Subquery?**
**Answer:** Subqueries run independently of the outer query. Correlated subqueries reference columns from the outer query and run once for each row in the outer query.

**Q31. Explain the difference between NULL, zero, and an empty string.**
**Answer:** NULL represents a missing or unknown value. Zero is a defined numeric value. An empty string is a defined text string with length 0.

**Q32. What does the UNION operator do? How does it differ from UNION ALL?**
**Answer:** Both combine result sets vertically. UNION removes duplicates (slow), while UNION ALL keeps all duplicate rows (fast).

**Q33. How do you find the Nth highest salary?**
**Answer:** Using a correlated subquery or using `DENSE_RANK() OVER (ORDER BY salary DESC)` inside a CTE, then filtering where rank = N.

**Q34. What is a CTE (Common Table Expression)? Why is it used over subqueries?**
**Answer:** A temporary named result set defined using the `WITH` clause. It improves query readability, is reusable within the query scope, and supports recursion.

**Q35. Explain window functions. How are they different from GROUP BY?**
**Answer:** GROUP BY collapses rows into a single summary row per group. Window functions calculate values over a partition of rows without collapsing them; every row remains distinct in the output.

**Q36. Compare RANK(), DENSE_RANK(), and ROW_NUMBER().**
**Answer:** `ROW_NUMBER` assigns sequential numbers. `RANK` assigns ranks, skipping numbers after ties. `DENSE_RANK` assigns consecutive ranks, without skipping numbers.

**Q37. How do you delete duplicate records keeping the one with the smallest ID?**
**Answer:** `DELETE FROM employees WHERE emp_id NOT IN (SELECT MIN(emp_id) FROM employees GROUP BY name, dept_id);`

**Q38. What are ACID properties?**
**Answer:** Atomicity (all-or-nothing), Consistency (rules preserved), Isolation (independent transactions), and Durability (committed changes survive crashes).

**Q39. What is a View? What are the benefits?**

**Answer:** A virtual table defined by a saved SQL query. It simplifies queries, provides security by restricting column access, and ensures logical data independence.

**Q40. What is an Index? Explain Clustered vs Non-Clustered.**
**Answer:** A B+ Tree data structure that speeds up data retrieval. Clustered indexes determine the physical storage order of rows (limit of 1). Non-clustered indexes store search keys and pointers to rows (multiple allowed).

**Q41. Explain the Write-Ahead Logging (WAL) protocol.**
**Answer:** Modifying a database page requires first writing the log entry describing the change to disk, guaranteeing crash recovery and durability.

**Q42. Why are B+ Trees preferred over B Trees for indexing?**
**Answer:** B+ Trees store data pointers only in leaf nodes (allowing internal nodes to hold more search keys, increasing fan-out and reducing height) and leaf nodes are linked, optimizing sequential scans and range queries.

**Q43. What is the CAP Theorem?**
**Answer:** A distributed system can simultaneously guarantee at most two of the following: Consistency, Availability, and Partition Tolerance. Under network partition, a system must trade off consistency or availability.

**Q44. What is a Deadlock? How does the DBMS handle it?**
**Answer:** A circular wait condition where transactions wait for locks held by each other. Handled using detection (Wait-For Graph cycles) or prevention (Wait-Die or Wound-Wait timestamp schemes).

**Q45. Explain the isolation levels defined in SQL.**
**Answer:** Read Uncommitted (allows dirty reads), Read Committed (prevents dirty reads), Repeatable Read (prevents non-repeatable reads), and Serializable (prevents all anomalies).

**Q46. What is the difference between a Stored Procedure and a Function?**
**Answer:** Stored procedures can perform write transactions and return multiple values/tables. Functions must return a single value and cannot perform write transactions (no COMMIT/ROLLBACK).

**Q47. Explain the difference between horizontal partitioning, vertical partitioning, and sharding.**
**Answer:** Horizontal partitioning splits rows on the same server. Vertical partitioning splits columns into separate tables on the same server. Sharding distributes partitions across multiple database servers.

**Q48. What is a Trigger?**
**Answer:** A database object that automatically executes in response to DML events (INSERT, UPDATE, DELETE).

**Q49. Compare OLTP and OLAP systems.**
**Answer:** OLTP is normalized (3NF), optimized for fast transaction writes/reads, and stores current data. OLAP is denormalized, optimized for complex analytics on large datasets, and stores historical data.

**Q50. Explain MongoDB collections and documents.**

**Answer:** Collections group documents (equivalent to tables in SQL). Documents store records as BSON objects (equivalent to rows in SQL).

**Q51. What is the difference between NVL, IFNULL, and COALESCE?**
**Answer:** `NVL` is Oracle-specific. `IFNULL` is MySQL-specific. `COALESCE` is the ANSI-SQL standard function that returns the first non-null argument.

**Q52. Explain the Wait-For Graph (WFG).**
**Answer:** A directed graph used to detect deadlocks. Nodes represent transactions, and edges represent lock wait dependencies. Cycles in the graph indicate deadlocks.

**Q53. What is a self-referencing foreign key?**
**Answer:** A foreign key column that points to the primary key of the same table (e.g., manager ID in an employee table).

**Q54. What are domain constraints?**
**Answer:** Constraints defining the set of valid values for a column (data types, lengths, and CHECK conditions).

**Q55. What is a checkpoint?**
**Answer:** An operation that writes all dirty memory buffers and logs to disk, limiting how far back the recovery manager must scan during crash recovery.

**Q56. What is a trivial functional dependency?**
**Answer:** $X \rightarrow Y$ where $Y$ is a subset of $X$ (e.g., `(EmpID, Name) -> EmpID`).

**Q57. What is an update anomaly?**
**Answer:** An inconsistency that occurs when duplicate data in one table is updated in one place but not another.

**Q58. What is a partial dependency?**
**Answer:** When a non-prime attribute functionally depends on a proper subset of a composite candidate key, violating 2NF.

**Q59. What is a transitive dependency?**
**Answer:** When a non-prime attribute functionally determines another non-prime attribute, violating 3NF.

**Q60. Why is BCNF stricter than 3NF?**
**Answer:** BCNF removes the "Y is a prime attribute" exception, requiring the LHS of all non-trivial FDs to be a super key.

**Q61. What is a Cartesian Product?**
**Answer:** The pairing of every row in one table with every row in another, produced by a CROSS JOIN.

**Q62. What does `LIMIT 1 OFFSET 2` do?**
**Answer:** Skips the first two rows and returns the third row.

**Q63. What is the result of `SELECT 10 / NULL`?**
**Answer:** NULL. Any arithmetic operation involving a NULL value yields NULL.

**Q64. Can you use aggregate functions in a WHERE clause?**
**Answer:** No. WHERE filters rows before aggregation occurs. Use HAVING to filter aggregate values.

**Q65. What is a recursive CTE?**
**Answer:** A CTE that references itself, used to query hierarchical data (e.g., organizational charts).

**Q66. What is the difference between CHAR and VARCHAR?**
**Answer:** CHAR is fixed-length, padding values with spaces. VARCHAR is variable-length, using only the bytes needed plus a length prefix.

**Q67. What does the `NTILE(n)` function do?**
**Answer:** Divides a sorted partition of rows into `n` roughly equal groups or buckets.

**Q68. What is a natural join?**
**Answer:** Joins tables automatically based on columns with identical names and data types, which can cause errors if columns share unrelated names.

**Q69. What is the default port for MySQL?**
**Answer:** 3306.

**Q70. What is the default port for PostgreSQL?**
**Answer:** 5432.

**Q71. What does the `%` wildcard match in a LIKE query?**
**Answer:** Zero or more characters.

**Q72. What does the `_` wildcard match in a LIKE query?**
**Answer:** Exactly one character.

**Q73. What does `SELECT 1` do in an EXISTS subquery?**
**Answer:** Evaluates the presence of matching rows, ignoring projected columns.

**Q74. What is denormalization?**
**Answer:** The process of introducing redundancy by combining tables to improve read performance in read-heavy applications (OLAP).

**Q75. What is a sharding key?**
**Answer:** The column used to distribute rows across physical servers in a sharded database.

**Q76. What is BSON?**
**Answer:** Binary JSON, the binary serialization format used to store documents in MongoDB.

**Q77. What is a cursor?**

**Answer:** A database object used to retrieve and process rows of a query result set one row at a time.

**Q78. What is the compatibility of two Shared (S) locks?**
**Answer:** Compatible. Multiple transactions can hold shared locks on the same row simultaneously.

**Q79. What is the compatibility of a Shared (S) lock and an Exclusive (X) lock?**
**Answer:** Conflicting. An exclusive lock blocks all other locks.

**Q80. What is a savepoint?**
**Answer:** A checkpoint within a transaction that allows rolling back specific statements without undoing the entire transaction.

**Q81. What is an active transaction state?**
**Answer:** The initial state where a transaction begins execution and read/write statements are processed.

**Q82. What is an aborted transaction state?**
**Answer:** The state entered after the database is rolled back to its pre-transaction state following a failure.

**Q83. Explain the Wait-Die scheme.**
**Answer:** Older transactions wait for younger ones. Younger transactions die (abort) when requesting locks held by older ones.

**Q84. Explain the Wound-Wait scheme.**
**Answer:** Older transactions wound (abort) younger ones to acquire locks. Younger transactions wait for older ones.

**Q85. What is strict 2PL?**
**Answer:** Requires all exclusive locks to be held until the transaction commits or aborts, preventing cascading rollbacks.

**Q86. What is rigorous 2PL?**
**Answer:** Requires all locks (both shared and exclusive) to be held until the transaction commits or aborts.

**Q87. What is a clustered index leaf node?**
**Answer:** Contains the actual physical data rows of the table.

**Q88. What is a non-clustered index leaf node?**
**Answer:** Contains search keys and row pointers to physical row locations.

**Q89. What is a cascading rollback?**
**Answer:** When a transaction failure requires rolling back multiple dependent transactions that read its uncommitted data.

**Q90. What is the intersection of two tables?**
**Answer:** The set of rows present in both tables, returned by the `INTERSECT` operator.

**Q91. What is the difference between UNION and UNION ALL?**
**Answer:** UNION removes duplicate rows (slow), while UNION ALL keeps all duplicate rows (fast).

**Q92. What does `SELECT NULL = NULL` evaluate to?**
**Answer:** NULL.

**Q93. What does the HAVING clause do?**
**Answer:** Filters aggregated groups after they are grouped by a `GROUP BY` clause.

**Q94. What is a derived table?**
**Answer:** A subquery defined in a FROM clause.

**Q95. What does `COALESCE(NULL, 10, NULL)` return?**
**Answer:** 10.

**Q96. Can a view be updated if it contains a GROUP BY clause?**
**Answer:** No. Views containing aggregations, groupings, or distinct filters are read-only.

**Q97. What is a primary key?**
**Answer:** The selected candidate key used to identify rows. Cannot contain NULLs.

**Q98. What is a candidate key?**
**Answer:** A minimal super key.

**Q99. What is a super key?**
**Answer:** Any set of columns that uniquely identifies a row.

**Q100. What is a transaction?**
**Answer:** A logical unit of database execution that must run as a single atomic unit.

---

## ■ Last-Minute 1-Hour Revision Notes ■■■■■

*Read this section in the final hour before your interview.*

- **ACID Guarantees:** Atomicity (all or nothing), Consistency (schema rules preserved), Isolation (concurrency boundaries), Durability (persistent writes).
- **Logical Execution Order:** FROM -> JOIN -> ON -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT.
- **Keys:**
  - *Super Key:* Uniquely identifies rows.
  - *Candidate Key:* Minimal super key.
  - *Primary Key:* Selected candidate key (cannot contain NULLs).
  - *Foreign Key:* Points to primary key in parent table (enforces referential integrity).
- **Normalization Rules:**
  - *1NF:* Atomic attributes.

- *2NF:* 1NF + No Partial Dependency ($\text{Part of Key} \rightarrow \text{Non-Prime}$).
- *3NF:* 2NF + No Transitive Dependency ($Key \rightarrow A \rightarrow B$).
- *BCNF:* LHS of all non-trivial FDs must be a Super Key.

• **Decomposition Tests:**

- *Lossless:* Intersection must be a key for $R_1$ or $R_2$ ($R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$).
- *Dependency Preserving:* All original FDs can be checked within individual sub-relations.

• **Locks:**

- *Shared (S):* Read lock. Multiple transactions can hold shared locks concurrently.
- *Exclusive (X):* Write lock. Blocks all other locks.
- *2PL:* Growing phase (acquire locks), Shrinking phase (release locks). Holds all exclusive locks until commit in Strict 2PL.

• **Crash Recovery:** Write-Ahead Logging (WAL) requires writing logs to disk before modifying data pages. Checkpoints write dirty buffers and logs to disk, limiting recovery scan times.

• **Indexes:**

- *Clustered:* Physical table rows are sorted in index order. Limit of one per table.
- *Non-Clustered:* Search keys and pointers to row locations. Multiple allowed.
- *B+ Tree:* Internal nodes store keys; leaf nodes store data pointers and are linked to optimize range scans.

• **SQL Window Functions:**

- `ROW_NUMBER()`: Consecutive integers.
- `RANK()`: Ranks with gaps.
- `DENSE_RANK()`: Ranks without gaps.
- `LAG()` / `LEAD()`: Access previous/next rows.

• **NoSQL & Scaling:**

- *OLTP:* High write concurrency, normalized.
- *OLAP:* Aggregations and analytics, denormalized.
- *Partitioning:* Table split on a single server.
- *Sharding:* Partitioning split across multiple servers.
- *CAP Theorem:* Network partitions require trading off consistency or availability.
- *MongoDB:* Document-oriented NoSQL database. Stores documents (BSON) in collections.